

SOLIDWORKS API (NET VBA C#) — Examples (50/50):

Undo_Deleted_Note_and_Fire_Undo_Post-Not- Zoom_to_Region_Example_VB

SOLIDWORKS API Documentation (converted from CHM)

Undo Deleted Note and Fire Undo Pre- and Post-notify Events Example (VB.NET)

This example demonstrates firing Undo pre- and post-notification events in a drawing document.

```
'-----  
' Preconditions: Open public_documents\samples\tutorial\AutoCAD\7550-021.slddrw.  
'-----  
' Postconditions:  
' 1. Selects and deletes the note in Drawing View1.  
' 2. Undoes the deleted note.  
' 3. Fires pre-notification event indicating that an undo action is about to  
'    occur and fires post-notification event indicating that an undo  
'    action occurred.  
' 4. Click OK to close each message box.  
'-----  
' NOTE: Because the drawing is used elsewhere, do not save changes.  
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Collections
```

```
Imports System.Windows.Forms
```

```
Partial Class SolidWorksMacro
```

```
    Public WithEvents swDrawing As DrawingDoc
```

```
    Public Sub main()
```

```
        Dim swModel As ModelDoc2
```

```
        Dim swModelDocExt As ModelDocExtension
```

```
        Dim boolstatus As Boolean
```

```
        Dim openDrawing As Hashtable
```

```
        swModel = swApp.ActiveDoc
```

```
        swModelDocExt = swModel.Extension
```

```
        ' Event notification
```

```
        swDrawing = swModel
```

```
        openDrawing = New Hashtable
```

```
        AttachEventhandlers()
```

```
        ' Activate the drawing view that contains
```

```
        ' the note you want to delete
```

```
        boolstatus = swDrawing.ActivateView("Drawing View3")
```

```
        boolstatus = swModelDocExt.SelectByID2("DetailItem77@Drawing View3", "NOTE",  
0.3058741216774, 0.1870419466786, 0, False, 0, Nothing, 0)
```

```
        ' Delete the selected note
```

```
swModel.EditDelete()
```

```
' Undo deletion of note
```

```
' Pre- and post-notifications fired
```

```
swModel.EditUndo2(1)
```

```
' Rebuild the drawing
```

```
swModel.ForceRebuild3(True)
```

```
End Sub
```

```
Sub AttachEventHandlers()
```

```
    AttachSWEvents()
```

```
End Sub
```

```
Sub AttachSWEvents()
```

```
    AddHandler swDrawing.UndoPostNotify, AddressOf Me.swDrawing_UndoPostNotify
```

```
    AddHandler swDrawing.UndoPreNotify, AddressOf Me.swDrawing_UndoPreNotify
```

```
End Sub
```

```
Private Function swDrawing_UndoPostNotify() As Integer
```

```
    ' Display message after Undo
```

```
    ' NOTE: Because the message box may be displayed
```

```
    ' behind an opened window, you might not see it.
```

```
    ' If so, then check the Taskbar.
```

```
    MsgBox("An undo post-notification event has been fired.")
```

```
End Function
```

```
Private Function swDrawing_UndoPreNotify() As Integer
```

```
    ' Display message after Undo
```

```
    ' NOTE: Because the message box may be displayed
```

```
    ' behind an opened window, you might not see it.
```

```
    ' If so, then check the Taskbar.
```

```
    MsgBox("An Undo pre-notification event has been fired.")
```

```
End Function
```

```
''' <summary>
```

```
''' The SldWorks swApp variable is pre-assigned for you.
```

```
''' </summary>
```

```
Public swApp As SldWorks
```

```
End Class
```

Undo Feature and Fire Undo Post-Notify Event Example (C#)

This example shows how to fire an undo post-notification in a part.

```
//-----  
// Preconditions: Open public_documents\samples\tutorial\api\cstick.sldprt.  
//  
// Postconditions:  
// 1. Creates a cut-extrude feature on the top face of the  
//    candlestick.  
// 2. Undoes the cut-extrude feature and fires an undo post-notification.  
// 3. Click OK to close the message box.  
// 4. Deletes the cut-extrude sketch and all absorbed features.  
//  
// NOTE: Because the part is used elsewhere, do not save changes.  
//-----  
  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Collections;  
using System.Windows.Forms;  
namespace UndoPostNotifyCSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        public PartDoc swPart;  
        public void Main()  
        {  
            ModelDoc2 swModel;  
            ModelDocExtension swModelDocExt;  
            SketchManager swSketchManager;  
            SketchSegment swSketchSegment;  
            FeatureManager swFeatureManager;  
            Feature swFeature;  
            bool boolstatus = false;  
  
            Hashtable openPart;  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
  
            // Set up event notification  
            swPart = (PartDoc)swModel;  
            openPart = new Hashtable();  
            AttachEventHandlers();  
  
            // Create a cut-extrude feature on the  
            // top face of the candlestick  
            swModelDocExt = swModel.Extension;
```

```

        boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.00140404215739, 0.21999999999999,
0.001897848026772, false, 0, null, 0);
        swSketchManager = swModel.SketchManager;
        swSketchSegment = swSketchManager.CreateCircle(0.0, 0.0, 0.0, 0.01296, -0.006347,
0.0);
        swModel.ClearSelection2(true);
        boolstatus = swModelDocExt.SelectByID2("Arc1", "SKETCHSEGMENT", 0, 0, 0, false, 0,
null, 0);
        swFeatureManager = swModel.FeatureManager;
        swFeature = swFeatureManager.FeatureCut(true, false, false, 0, 0, 0.01, 0.01, false,
false, false,
true
        false, 0.01745329251994, 0.01745329251994, false, false, false, false, false, true,
        );
        swModel.ClearSelection2(true);

        // Undo the cut-extrude feature
        swModel.EditUndo2(1);

        // Fire undo notification

        // Select the circle and delete it
        boolstatus = swModelDocExt.SelectByID2("Arc1", "SKETCHSEGMENT", 0, 0, 0, false, 0,
null, 0);
        boolstatus =
swModelDocExt.DeleteSelection2((int)swDeleteSelectionOptions_e.swDelete_Absorbed);

        swModel.ForceRebuild3(true);
    }

    public void AttachEventHandlers()
    {
        AttachSWEvents();
    }

    public void AttachSWEvents()
    {
        swPart.UndoPostNotify += this.swPart_UndoPostNotify;
    }

    public int swPart_UndoPostNotify()
    {
        //Show message after undo action occurs
        // NOTE: Because the message box might be displayed
        // behind an opened window, you might not see it.
        // If so, then check the Taskbar.
        MessageBox.Show("An undo post-notification event has been fired.");
        return 1;
    }
}

```

```
    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}
```

Undo Feature and Fire Undo Post-Notify Event Example (VBA)

This example shows how to fire an undo post-notification in a part.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\cstick.sldprt.  
' 2. Copy and paste Main in the main module.  
' 3. Click Insert > Class Module and copy and paste Class1 into that module.  
'  
' Postconditions:  
' 1. Creates a cut-extrude feature on the top face of the  
'   candlestick.  
' 2. Undoes the cut-extrude feature and fires an undo post-notification.  
' 3. Click OK to close the message box.  
' 4. Deletes the cut-extrude sketch and all absorbed features.  
'  
' NOTE: Because the part is used elsewhere, do not save changes.  
'-----  
'Main  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swPart As SldWorks.PartDoc  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSketchManager As SldWorks.SketchManager  
Dim swSketchSegment As SldWorks.SketchSegment  
Dim swFeatureManager As SldWorks.FeatureManager  
Dim swFeature As SldWorks.Feature  
Dim boolstatus As Boolean  
Dim swPartEvents As Class1  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    ' Event notification  
    Set swPart = swModel  
    Set swPartEvents = New Class1  
    Set swPartEvents.swPart = swApp.ActiveDoc  
  
    ' Create a cut-extrude feature on the  
    ' top face of the candlestick  
    Set swModelDocExt = swModel.Extension  
    boolstatus = swModelDocExt.SelectByID("", "FACE", 0.00140404215739, 0.219999999999999, 0.001897848026772,  
False, 0, Nothing, 0)  
    Set swSketchManager = swModel.SketchManager  
    Set swSketchSegment = swSketchManager.CreateCircle(0#, 0#, 0#, 0.01296, -0.006347, 0#)  
    swModel.ClearSelection2 True  
    boolstatus = swModelDocExt.SelectByID("Arc1", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)  
    Set swFeatureManager = swModel.FeatureManager  
    Set swFeature = swFeatureManager.FeatureCut(True, False, False, 0, 0, 0.01, 0.01, False, False, False, False,  
0.01745329251994, 0.01745329251994, False, False, False, False, False, True, True)  
    swModel.ClearSelection2 True  
  
    ' Undo the cut-extrude feature  
    swModel.EditUndo2 1  
  
    ' Fire undo event  
  
    ' Select the circle and delete it  
    boolstatus = swModelDocExt.SelectByID("Arc1", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)  
    boolstatus = swModelDocExt.DeleteSelection2(swDelete_Absorbed)  
  
    swModel.ForceRebuild3 True  
  
End Sub
```

[Back to top](#)

```
'Class1
Option Explicit

Public WithEvents swPart As SldWorks.PartDoc

Private Function swPart_UndoPostNotify() As Long
    'Show message after an undo action occurs
    MsgBox "An undo post-notification event has been fired."
End Function
```

[Back to top](#)

Undo Feature and Fire Undo Post-Notify Event (VB.NET)

This example shows how to fire an undo post-notification in a part.

```
'-----  
' Preconditions: Open public_documents\samples\tutorial\api\cstick.sldprt.  
'  
' Postconditions:  
' 1. Creates a cut-extrude feature on the top face of the  
'    candlestick.  
' 2. Undoes the cut-extrude feature and fires an undo post-notification.  
' 3. Click OK to close the message box.  
' 4. Deletes the cut-extrude sketch and all absorbed features.  
'  
' NOTE: Because the part is used elsewhere, do not save changes.  
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Collections
```

```
Partial Class SolidWorksMacro
```

```
    Public WithEvents swPart As PartDoc
```

```
    Public Sub Main()
```

```
        Dim swModel As ModelDoc2
```

```
        Dim swModelDocExt As ModelDocExtension
```

```
        Dim swSketchManager As SketchManager
```

```
        Dim swSketchSegment As SketchSegment
```

```
        Dim swFeatureManager As FeatureManager
```

```
        Dim swFeature As Feature
```

```
        Dim boolstatus As Boolean
```

```
        Dim openPart As Hashtable
```

```
        swModel = swApp.ActiveDoc
```

```
        ' Set up event notification
```

```
        swPart = swModel
```

```
        openPart = New Hashtable
```

```
        AttachEventHandlers()
```

```
        ' Create a cut-extrude feature on the
```

```
        ' top face of the candlestick
```

```
        swModelDocExt = swModel.Extension
```

```

        boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.00140404215739, 0.21999999999999,
0.001897848026772, False, 0, Nothing, 0)
        swSketchManager = swModel.SketchManager
        swSketchSegment = swSketchManager.CreateCircle(0.0#, 0.0#, 0.0#, 0.01296, -0.006347, 0.0#)
        swModel.ClearSelection2(True)
        boolstatus = swModelDocExt.SelectByID2("Arc1", "SKETCHSEGMENT", 0, 0, 0, False, 0,
Nothing, 0)
        swFeatureManager = swModel.FeatureManager
        swFeature = swFeatureManager.FeatureCut(True, False, False, 0, 0, 0.01, 0.01, False,
False, False, False, 0.01745329251994, 0.01745329251994, False, False, False, False, True,
True)

        swModel.ClearSelection2(True)

        ' Undo the cut-extrude feature
        swModel.EditUndo2(1)

        ' Fire undo notification

        ' Select the circle and delete it
        boolstatus = swModelDocExt.SelectByID2("Arc1", "SKETCHSEGMENT", 0, 0, 0, False, 0,
Nothing, 0)
        boolstatus = swModelDocExt.DeleteSelection2(swDeleteSelectionOptions_e.swDelete_Absorbed)

        swModel.ForceRebuild3(True)

End Sub

Sub AttachEventHandlers()
    AttachSWEvents()
End Sub

Sub AttachSWEvents()
    AddHandler swPart.UndoPostNotify, AddressOf Me.swPart_UndoPostNotify
End Sub

Function swPart_UndoPostNotify() As Integer
    ' Show message after undo action occurs
    ' NOTE: Because the message box might be displayed
    ' behind an open window, you might not see it.
    ' If so, then check the Taskbar.
    MsgBox("An undo post-notification event has been fired.")
End Function

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Undo Hidden Component and Fire Undo Post-Notify Event Example (C#)

This example shows how to fire an undo post-notification in an assembly document.

```
//-----  
// Preconditions: Open public_documents\samples\tutorial\smartcomponents\stepped_shaft.sldasm.  
//  
// Postconditions:  
// 1. Selects and hides the base-plate<1> component.  
// 2. Undoes the hiding of the base-plate<1> component.  
// 3. Fires a post-notification indicating that an undo action occurred.  
// 4. Click OK to close the message box.  
//  
// NOTE: Because the assembly is used elsewhere, do not save changes.  
//-----  
using SolidWorks.Interop.sldworks;  
  
using SolidWorks.Interop.swconst;  
using System;  
using System.Collections;  
using System.Windows.Forms;  
namespace Macro1.csproj  
{  
    public partial class SolidWorksMacro  
    {  
        public AssemblyDoc swAssemblyDoc;  
  
        public void Main()  
        {  
            ModelDoc2 swModel = default(ModelDoc2);  
            ModelDocExtension swModelDocExt = default(ModelDocExtension);  
            bool boolstatus = false;  
            Hashtable openAssembly = default(Hashtable);  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            swModelDocExt = (ModelDocExtension)swModel.Extension;  
  
            // Event notification  
            swAssemblyDoc = (AssemblyDoc)swModel;  
            openAssembly = new Hashtable();  
            AttachEventHandlers();  
  
            // Select a component and hide it  
            boolstatus = swModelDocExt.SelectByID2("base_plate-1@stepped_shaft", "COMPONENT", 0,  
0, 0, false, 0, null, 0);  
            swModel.HideComponent2();  
            swModel.ClearSelection2(true);  
  
            // Undo hiding of the component
```

```

        swModel.EditUndo2(1);

        // Fire undo post-notiifcation

        // Rebuild model
        swModel.ForceRebuild3(true);
    }
    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;

    public void AttachEventHandlers()
    {
        AttachSWEvents();
    }

    public void AttachSWEvents()
    {
        swAssemblyDoc.UndoPostNotify += this.swAssembly_UndoPostNotify;
    }

    public int swAssembly_UndoPostNotify()
    {
        //Display message after undo action occurs
        MessageBox.Show("An undo post-notification event has been fired.");
        return 1;
    }
}
}

```

Undo Hidden Component and Fire Undo Post-Notify Event Example (VBA)

This example shows how to fire an undo post-notification in an assembly document.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\smartcomponents\stepped_shaft.sldasm.
' 2. Copy and paste Main into the main module.
' 3. Click Insert > Class Module and copy and paste Class1 into that module.
'
' Postconditions:
' 1. Selects and hides the base-plate<1> component.
' 2. Undoes the hiding of the base-plate<1> component.
' 3. Fires a post-notification indicating that an undo action occurred.
' 4. Click OK to close the message box.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
'Main
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swAssembly As SldWorks.AssemblyDoc
Dim boolstatus As Boolean
Dim swAssemblyEvents As Class1

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    ' Event notification
    Set swAssembly = swModel
    Set swAssemblyEvents = New Class1
    Set swAssemblyEvents.swAssembly = swApp.ActiveDoc

    ' Select a component and hide it
    boolstatus = swModelDocExt.SelectByID2("base_plate-1@stepped_shaft", "COMPONENT", 0, 0, 0, False, 0, Nothing,
0)
    swModel.HideComponent2

    swModel.ClearSelection2 True

    ' Undo hiding of the component
    swModel.EditUndo2 1

    ' Fire undo post-notification

    'Rebuild the assembly
    swModel.ForceRebuild3 True

End Sub

'-----
'Class 1
Option Explicit

Public WithEvents swAssembly As SldWorks.AssemblyDoc

Private Function swAssembly_UndoPostNotify() As Long
    'Display message after undo action occurs
    MsgBox "An undo post-notification event has been fired."
End Function
```

[Go to top](#)

[Go to top](#)

Undo Hidden Component and Fire Undo Post-Notify Event Example (VB.NET)

This example shows how to fire an undo post-notification in an assembly document.

```
' -----  
' Preconditions: Open public_documents\samples\tutorial\smartcomponents\stepped_shaft.sldasm.  
'  
' Postconditions:  
' 1. Selects and hides the base-plate<1> component.  
' 2. Undoes the hiding of the base-plate<1> component.  
' 3. Fires a post-notification indicating that an undo action occurred.  
' 4. Click OK to close the message box.  
'  
' NOTE: Because the assembly is used elsewhere, do not save changes.  
' -----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Collections
```

```
Imports System.Windows.Forms
```

```
Partial Class SolidWorksMacro
```

```
    Public WithEvents swAssemblyDoc As AssemblyDoc
```

```
    Public Sub main()
```

```
        Dim swModel As ModelDoc2
```

```
        Dim swModelDocExt As ModelDocExtension
```

```
        Dim boolstatus As Boolean
```

```
        Dim openAssembly As Hashtable
```

```
        swModel = swApp.ActiveDoc
```

```
        swModelDocExt = swModel.Extension
```

```
        ' Event notification
```

```
        swAssemblyDoc = swModel
```

```
        openAssembly = New Hashtable
```

```
        AttachEventHandlers()
```

```
        ' Select a component and hide it
```

```
        boolstatus = swModelDocExt.SelectByID2("base_plate-1@stepped_shaft", "COMPONENT", 0, 0, 0,  
False, 0, Nothing, 0)
```

```
        swModel.HideComponent2()
```

```
        swModel.ClearSelection2(True)
```

```
        ' Undo hiding of the component
```

```
        swModel.EditUndo2(1)
```

```

        ' Fire undo post-notification

        ' Rebuild the model
        swModel.ForceRebuild3(True)

End Sub

Sub AttachEventHandlers()
    AttachSWEvents()
End Sub

Sub AttachSWEvents()
    AddHandler swAssemblyDoc.UndoPostNotify, AddressOf Me.swAssembly_UndoPostNotify
End Sub

Private Function swAssembly_UndoPostNotify() As Integer
    'Display message after undo event occurs
    MsgBox("An undo post-notification event has been fired.")
End Function

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Unlink Dimensions Example (VBA)

This example shows how to unlink dimensions in a part.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\lesson2\tutor1.sldprt.  
' 2. Right-click Annotations and click Show Feature Dimensions.  
' 3. Right-click either 120 dimension and click Link Values.  
' 4. Type AreaDimension and click OK.  
' 5. Right-click the other 120 dimension and click Link Values.  
' 6. Type AreaDimension and click OK.  
' 7. Open the Immediate window.  
'  
' Postconditions:  
' 1. Removes the links from the linked dimensions.  
' 2. Examine the Immediate window and graphics area.  
'  
' NOTE:  
' * This macro gets all dimensions in the part, including dimensions  
'   that are not associated with a feature. These types of  
'   dimensions usually appear under Annotations.  
' * Because the part is used elsewhere, do not save changes.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swFeat As SldWorks.Feature  
    Dim swDimen As SldWorks.Dimension  
    Dim swDispDim As SldWorks.DisplayDimension  
    Dim swDispDimAnn As SldWorks.Annotation  
    Dim nRetval As Long  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swFeat = swModel.FirstFeature  
  
    Debug.Print "File = " & swModel.GetPathName  
  
    Do While Not swFeat Is Nothing  
        Set swDispDim = swFeat.GetFirstDisplayDimension  
        If Not swDispDim Is Nothing Then  
            Debug.Print " " & swFeat.Name  
            End If  
            Do While Not swDispDim Is Nothing  
                Set swDispDimAnn = swDispDim.GetAnnotation  
                Set swDimen = swDispDim.GetDimension  
                If swDispDim.IsLinked Then  
                    Debug.Print " " & swDispDimAnn.GetName & " [" & swDimen.FullName & "] -- > " &  
swDispDim.GetLinkedText  
                    nRetval = swDispDim.Unlink: Debug.Assert swLinkDimensionError_NoError = nRetval  
                    End If  
                    Set swDispDim = swFeat.GetNextDisplayDimension(swDispDim)  
                Loop  
                Set swFeat = swFeat.GetNextFeature  
            Loop  
        End Sub
```


Update_All_Toolbox_Components_Example_CSharp (Topic)

SOLIDWORKS API Help

Update All Toolbox Components Example (C#)

This example shows how to update all SOLIDWORKS Toolbox components in an assembly.

```
//-----  
// Preconditions:  
// 1. Verify that the specified assembly exists.  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Updates all SOLIDWORKS Toolbox components in the assembly.  
// 2. Examine the Immediate window.  
//  
// NOTE: Because this assembly is used elsewhere, do not save  
// changes.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
  
namespace Macro1.CSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            ModelDoc2 swModel = default(ModelDoc2);  
            AssemblyDoc swAssembly = default(AssemblyDoc);  
            int errors = 0;  
            int warnings = 0;  
            int updateToolboxComponents = 0;  
  
            // Open the assembly  
            swModel = (ModelDoc2)swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS  
2018\\samples\\tutorial\\toolbox\\lens_mount.sldasm", (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e  
.swOpenDocOptions_Silent, "", ref errors, ref warnings);  
            swAssembly = (AssemblyDoc)swModel;  
  
            // Update all SOLIDWORKS Toolbox components in the assembly  
            updateToolboxComponents = swAssembly.UpdateToolboxComponent((int)swAssemblyLevelToUpdate_e.swAssemblyLe  
velToUpdate_AllLevels);  
            Debug.Print("Update SOLIDWORKS Toolbox components status: " + updateToolboxComponents);  
  
        }  
  
        /// <summary>  
        /// The SldWorks swApp variable is pre-assigned for you.  
        /// </summary>  
        public SldWorks swApp;  
    }  
}
```

Update All Toolbox Components Example (VBA)

This example shows how to update all SOLIDWORKS Toolbox components in an assembly.

```
'-----  
' Preconditions:  
' 1. Verify that the specified assembly exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Updates all SOLIDWORKS Toolbox components in the assembly.  
' 2. Examine the Immediate window.  
'  
' NOTE: Because the assembly is used elsewhere, do not save  
' changes.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swAssembly As SldWorks.AssemblyDoc  
Dim errors As Long  
Dim warnings As Long  
Dim updateToolboxComponents As Long  
  
Sub main()  
    Set swApp = Application.SldWorks  
    ' Open the assembly  
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\toolbox\lens_mount.sldasm", swDocumentTypes_e.swDocASSEMBLY,  
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)  
    Set swAssembly = swModel  
  
    ' Update all SOLIDWORKS Toolbox components in the assembly  
    updateToolboxComponents =  
swAssembly.updateToolboxComponent(swAssemblyLevelToUpdate_e.swAssemblyLevelToUpdate_AllLevels)  
    Debug.Print ("Update SOLIDWORKS Toolbox components status: " & updateToolboxComponents)  
  
End Sub
```

Update_All_Toolbox_Components_Example_VBNET (Topic)

SOLIDWORKS API Help

Update All Toolbox Components Example (VB.NET)

This example shows how to update all SOLIDWORKS Toolbox components in an assembly.

```
'-----
' Preconditions:
' 1. Verify that the specified assembly exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Updates all SOLIDWORKS Toolbox components in the assembly.
' 2. Examine the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save
' changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swAssembly As AssemblyDoc
        Dim errors As Integer
        Dim warnings As Integer
        Dim updateToolboxComponents As Integer

        ' Open the assembly
        swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\toolbox\lens_mount.sldasm", swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOpti
ons_Silent, "", errors, warnings)
        swAssembly = swModel

        ' Update all SOLIDWORKS Toolbox components in the assembly
        updateToolboxComponents = swAssembly.UpdateToolboxComponent(swAssemblyLevelToUpdate_e.swAssemblyLevelToUpda
te_AllLevels)
        Debug.Print("Update SOLIDWORKS Toolbox components status: " & updateToolboxComponents)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Update FeatureManager Design Tree Example (VBA)

This example shows how to update the FeatureManager design tree.

```
'-----  
' Preconditions:  
' 1. Open a part.  
' 2. Select a feature.  
'  
' Postconditions:  
' 1. Adds a comment to the selected feature.  
' 2. At Stop, examine the FeatureManager design tree  
'    to verify that a Comments folder is not shown in  
'    the FeatureManager design tree, then press F5.  
' 3. Updates the FeatureManager design tree.  
' 4. Examine the FeatureManager design tree  
'    again to verify that the Comments folder  
'    is shown in the FeatureManager design tree.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swFeatMgr As SldWorks.FeatureManager  
    Dim swFeature As SldWorks.Feature  
    Dim swSelectionMgr As SldWorks.SelectionMgr  
    Dim swComment As SldWorks.Comment  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelectionMgr = swModel.SelectionManager  
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)  
    Set swComment = swFeature.AddComment("Comment for feature")  
  
    Stop  
    'Examine the FeatureManager design tree  
    'to verify that a Comments folder is not shown in  
    'the FeatureManager design tree, then press F5  
  
    Set swFeatMgr = swModel.FeatureManager  
    swFeatMgr.UpdateFeatureTree  
  
End Sub
```

Update Graphics Area After Changing Render Options Example (C#)

This example shows how to update the graphics area after changing the ray-trace rendering engine's options.

```
//-----
// Preconditions:
// 1. Verify that:
//    * specified part exists.
//    * ray-trace rendering engine is loaded in SOLIDWORKS Design.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. If prompted to use perspective views in rendering, click
//    Continue without camera or perspective and OK.
// 2. Changes the specified render options and updates the graphics area.
// 3. Examine the graphics area and Immediate window.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using System;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace RayTraceRendererCSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        RayTraceRenderer swRayTraceRenderer;
        RayTraceRendererOptions swRayTraceRenderOptions;
        int errors;
        int warnings;
        string fileName;

        public void Main()
        {
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\routing-
pipes\\fittings\\filter.sldprt";
            swModel = swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocO
ptions_Silent, "", ref errors, ref warnings);

            // Access ray-trace rendering engine
            //
            swRayTraceRenderer = (RayTraceRenderer)swApp.GetRayTraceRenderer((int)swRayTraceRenderType_e.ray_trace_renderer_mem
ber);

            // Get and set render options
            swRayTraceRenderOptions = swRayTraceRenderer.RayTraceRendererOptions;

            // Get current render values
            Debug.Print("Current render values");
            Debug.Print("  Contour/cartoon enabled? " + swRayTraceRenderOptions.ContourCartoonRenderingEnabled);
            Debug.Print("  Render type (0 = cartoon; 1 = contour) = " + swRayTraceRenderOptions.RenderType);
            Debug.Print(" ");

            // Change render values and update graphics area
            Debug.Print("Changed render values");
            swRayTraceRenderOptions.ContourCartoonRenderingEnabled = true;
            Debug.Print("  Contour/cartoon enabled? " + swRayTraceRenderOptions.ContourCartoonRenderingEnabled);
            swRayTraceRenderOptions.RenderType = (int)swRayTraceRenderingType_e.swRayTraceCartoon;
            Debug.Print("  Render type (0 = cartoon; 1 = contour) = " + swRayTraceRenderOptions.RenderType);
            swRayTraceRenderOptions.HasCartoonEdges = true;
            Debug.Print("  Has cartoon edges? " + swRayTraceRenderOptions.HasCartoonEdges);
            swRayTraceRenderOptions.UpdateGraphics();
        }

        public SldWorks swApp;
    }
}
```

Update Graphics Area After Changing Render Options Example (VBA)

This example shows how to update the graphics area after changing the ray-trace rendering engine's options.

```
'-----  
' Preconditions:  
' 1. Verify that:  
'    * specified part exists.  
'    * ray-trace rendering engine is loaded in SOLIDWORKS Design.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. If prompted to use perspective views in rendering, click  
'    Continue without camera or perspective and OK.  
' 2. Changes the specified render options and updates the graphics area.  
' 3. Examine the graphics area and Immediate window.  
'  
' NOTE: Because the part is used elsewhere, do not save changes.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swRayTraceRenderer As SldWorks.RayTraceRenderer  
Dim swRayTraceRenderOptions As SldWorks.RayTraceRenderOptions  
Dim errors As Long  
Dim warnings As Long  
Dim fileName As String  
  
Sub main()  
    Set swApp = Application.SldWorks  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\routing-  
pipes\fittings\filter.sldprt"  
    Set swModel = swApp.OpenDoc6(fileName, swDocPART, swOpenDocOptions_Silent, "", errors, warnings)  
  
    ' Access ray-trace rendering engine  
    Set swRayTraceRenderer = swApp.GetRayTraceRenderer(0)  
  
    ' Get and set render options  
    Set swRayTraceRenderOptions = swRayTraceRenderer.RayTraceRenderOptions  
  
    ' Get current render values  
    Debug.Print "Current render values"  
    Debug.Print "  Contour/cartoon enabled? " & swRayTraceRenderOptions.ContourCartoonRenderingEnabled  
    Debug.Print "  Render type (0 = cartoon; 1 = contour) = " & swRayTraceRenderOptions.RenderType  
    Debug.Print " "  
  
    ' Change render values and update graphics area  
    Debug.Print "Changed render values"  
    swRayTraceRenderOptions.ContourCartoonRenderingEnabled = True  
    Debug.Print "  Contour/cartoon enabled? " & swRayTraceRenderOptions.ContourCartoonRenderingEnabled  
    swRayTraceRenderOptions.RenderType = swRayTraceRenderingType_e.swRayTraceCartoon  
    Debug.Print "  Render type (0 = cartoon; 1 = contour) = " & swRayTraceRenderOptions.RenderType  
    swRayTraceRenderOptions.HasCartoonEdges = True  
    Debug.Print "  Has cartoon edges? " & swRayTraceRenderOptions.HasCartoonEdges  
    swRayTraceRenderOptions.UpdateGraphics  
  
End Sub
```

Update Graphics Area After Changing Render Options Example (VB.NET)

This example shows how to update the graphics area after changing the ray-trace rendering engine's options.

```

'-----
' Preconditions:
' 1. Verify that:
'   * specified part exists.
'   * ray-trace rendering engine is loaded in SOLIDWORKS Design.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. If prompted to use perspective views in rendering, click
'   Continue without camera or perspective and OK.
' 2. Changes the specified render options and updates the graphics area.
' 3. Examine the graphics area and Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swRayTraceRenderer As RayTraceRenderer
        Dim swRayTraceRenderOptions As RayTraceRenderOptions
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\routing-
pipes\fittings\filter.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent,
"", errors, warnings)

        ' Access ray-trace rendering engine
        swRayTraceRenderer = swApp.GetRayTraceRenderer(swRayTraceRenderType_e.ray_trace_rendering_engine_member)

        ' Get and set render options
        swRayTraceRenderOptions = swRayTraceRenderer.RayTraceRenderOptions

        ' Get current render values
        Debug.Print("Current render values")
        Debug.Print("  Contour/cartoon enabled? " & swRayTraceRenderOptions.ContourCartoonRenderingEnabled)
        Debug.Print("  Render type (0 = cartoon; 1 = contour) = " & swRayTraceRenderOptions.RenderType)
        Debug.Print(" ")

        ' Change render values and update graphics area
        Debug.Print("Changed render values")
        swRayTraceRenderOptions.ContourCartoonRenderingEnabled = True
        Debug.Print("  Contour/cartoon enabled? " & swRayTraceRenderOptions.ContourCartoonRenderingEnabled)
        swRayTraceRenderOptions.RenderType = swRayTraceRenderType_e.swRayTraceCartoon
        Debug.Print("  Render type (0 = cartoon; 1 = contour) = " & swRayTraceRenderOptions.RenderType)
        swRayTraceRenderOptions.HasCartoonEdges = True
        Debug.Print("  Has cartoon edges? " & swRayTraceRenderOptions.HasCartoonEdges)
        swRayTraceRenderOptions.UpdateGraphics()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Update Notes Without Activating Sheets Example (VBA)

This example shows how to update the notes in a drawing document without activating the drawing sheets.

```
'-----
' Preconditions:
' 1. Open a drawing document that has multiple
'    sheets that have multiple drawing views.
' 2. Insert a "Note" note in a drawing view
'    on one or more one drawing sheets.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Traverses:
'    a. features, getting drawing sheets only.
'    b. subfeatures, getting the drawing views.
'    c. annotations in the drawing views, getting
'       the notes and appending the letter Z to
'       the end of all "Note" notes in all drawing views
'       in all drawing sheets.
' 2. To verify step 1c, examine the Immediate window and
'    open each sheet that contained a "Note" note and
'    click that note.
'-----
Option Explicit

' From swstrings.bas
Public Const swTnDrSheet As String = "DrSheet"
Public Const swTnAbsoluteView As String = "AbsoluteView"
Public Const swTnDetailView As String = "DetailView"
Public Const swTnRelativeView As String = "RelativeView"
Public Const swTnSectionPartView As String = "SectionPartView"
Public Const swTnSectionAssemView As String = "SectionAssemView"
Public Const swTnUnfoldedView As String = "UnfoldedView"
Public Const swTnAuxiliaryView As String = "AuxiliaryView"

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim myModel As SldWorks.ModelDoc2
    Dim myDrawing As SldWorks.DrawingDoc
    Dim curFeature As SldWorks.Feature
    Dim featureTypeName As String
    Dim thisSheet As SldWorks.Sheet

    Set swApp = Application.SldWorks
    Set myModel = swApp.ActiveDoc
    Set myDrawing = myModel

    Dim boolstatus As Boolean

' Loop through the features in the document, looking at the drawings sheets only
    Set curFeature = myModel.FirstFeature
    While Not curFeature Is Nothing
        featureTypeName = curFeature.GetTypeName()
        If (featureTypeName = swTnDrSheet) Then
            Set thisSheet = curFeature.GetSpecificFeature2()
            Debug.Print thisSheet.GetName()
            ' Test the sheet feature by getting and editing
            ' certain notes on any sheet, active or not
            Call test_sheet_feature(curFeature)
            ' Test the sheet feature by using it to activate the sheet.
            ' boolstatus = myDrawing.ActivateSheet(thisSheet.GetName)
        End If
        Set curFeature = curFeature.GetNextFeature
    Wend

End Sub
```



```

Sub test_sheet_feature(thisSheetFeature As SldWorks.Feature)
    Dim curFeature As SldWorks.Feature
    Dim featureTypeName As String
    Dim thisView As SldWorks.View
    Dim viewName As String
    Dim curAnno As SldWorks.Annotation
    Dim annoType As Long
    Dim thisNote As SldWorks.Note
    Dim noteText As String, newNoteText As String, noteText2 As String
    Dim boolstatus As Boolean

    ' Continue traversing through the subfeatures of this sheet feature
    ' The drawing views should show up as subfeatures of the sheet feature
    Set curFeature = thisSheetFeature.GetFirstSubFeature()
    While Not curFeature Is Nothing
        featureTypeName = curFeature.GetType()
        If (featureTypeName = swTnAbsoluteView Or featureTypeName = swTnDetailView Or featureTypeName =
swTnRelativeView Or featureTypeName = swTnSectionPartView Or featureTypeName = swTnSectionAssemView Or
featureTypeName = swTnUnfoldedView Or featureTypeName = swTnAuxiliaryView) Then
            Set thisView = curFeature.GetSpecificFeature2()
            ' Loop through the annotations in this view, specifically looking for notes
            If Not thisView Is Nothing Then
                Debug.Print " " & thisView.GetName2()
                Set curAnno = thisView.GetFirstAnnotation3()
                While Not curAnno Is Nothing
                    annoType = curAnno.GetType()
                    If annoType = SwConst.swAnnotationType_e.swNote Then
                        Set thisNote = curAnno.GetSpecificAnnotation()
                        If Not thisNote Is Nothing Then
                            ' Do something with this note; make sure that it seems to be completely
                            ' accessible even if the sheet is not the current sheet
                            noteText = thisNote.GetText()
                            If (Left(noteText, 4) = "Note") Then
                                newNoteText = noteText & "Z"
                                boolstatus = thisNote.SetText(newNoteText)
                            End If
                            noteText2 = thisNote.GetText()
                            Debug.Print " " & noteText & " - " & noteText2
                        End If
                    End If
                    Set curAnno = curAnno.GetNext3()
                Wend
            End If
        End If
        Set curFeature = curFeature.GetNextSubFeature()
    Wend
End Sub

```

Update Plane Example (C#)

This example shows how to update a reference plane so that it is parallel to the screen.

```
//-----
// Preconditions: Verify that the specified part template exists.
//
// Postconditions:
// 1. Creates Boss-Extrude1 and Plane1.
// 2. Rotates Boss-Extrude1.
// 3. Examine Plane1 in the graphics area.
// 4. Press F5.
// 5. Updates Plane1 so that it is parallel to the screen.
// 6. Examine Plane1 in the graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchManager = default(SketchManager);
            SketchSegment swSketchSegment = default(SketchSegment);
            FeatureManager swFeatureManager = default(FeatureManager);
            Feature swFeature = default(Feature);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            RefPlane swRefPlane = default(RefPlane);
            ModelView swModelView = default(ModelView);
            RefPlaneFeatureData swRefPlaneFeatureData = default(RefPlaneFeatureData);
            bool status = false;

            //Create Boss-Extrude1 and Plane1
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2016\\templates\\part.p
            rtdot", 0, 0, 0);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swSketchManager = (SketchManager)swModel.SketchManager;
            swSketchManager.InsertSketch(true);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(-0.049503, 0.030205, 0.0, -0.049503, 0.0, 0
            .0);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(-0.049503, 0.0, 0.0, 0.0, 0.0, 0.0);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(0.0, 0.0, 0.0, 0.0, 0.019088, 0.0);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(0.0, 0.019088, 0.0, -0.03503, 0.037127, 0.0
            );
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(-0.03503, 0.037127, 0.0, -0.049503, 0.03020
            5, 0.0);
            swSketchManager.InsertSketch(true);
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swFeature = (Feature)swFeatureManager.FeatureExtrusion3(true, false, false, 0, 0, 0.04, 0.01, false, fa
            lse, false,
            false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, true, true, true,
            0, 0, false);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swSelectionMgr.EnableContourSelection = false;
            status = swModelDocExt.SelectByID2("", "VERTEX", -0.0350298017733792, 0.0371273946939409, 0.04, true, 0
            , null, 0);
            swRefPlane = (RefPlane)swFeatureManager.InsertRefPlane(4096, 0, 0, 0, 0, 0);
            swModel.ClearSelection2(true);

            //Rotate Boss-Extrude1
            status = swModelDocExt.SelectByID2("Boss-Extrude1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swModelView = (ModelView)swModel.ActiveView;
            swModelView.RotateAboutCenter(0, 0.00828812132228465);
            swModelView = (ModelView)swModel.ActiveView;
            swModelView.RotateAboutCenter(0, 0.00828812132228465);
            swModelView = (ModelView)swModel.ActiveView;
            swModelView.RotateAboutCenter(0.00576690959137988, 0);
            swModelView = (ModelView)swModel.ActiveView;
            swModelView.RotateAboutCenter(0.00576690959137988, 0.0165762426445693);
            swModelView = (ModelView)swModel.ActiveView;
            swModelView.RotateAboutCenter(0.00576690959137988, 0.00828812132228465);
            swModelView = (ModelView)swModel.ActiveView;
```

```

swModelView.RotateAboutCenter(0, 0.0165762426445693);
swModelView = (ModelView)swModel.ActiveView;
swModelView.RotateAboutCenter(0.0115338191827598, 0.0248643639668539);
swModelView = (ModelView)swModel.ActiveView;
swModelView.RotateAboutCenter(0.0115338191827598, 0.0331524852891386);
swModelView = (ModelView)swModel.ActiveView;
swModelView.RotateAboutCenter(0.00576690959137988, 0.0248643639668539);
swModelView = (ModelView)swModel.ActiveView;
swModelView.RotateAboutCenter(0.00576690959137988, 0.0165762426445693);
swModelView = (ModelView)swModel.ActiveView;
swModelView.RotateAboutCenter(0, 0.0248643639668539);
swModelView = (ModelView)swModel.ActiveView;
swModelView.RotateAboutCenter(0.00576690959137988, 0.0165762426445693);
swModelView = (ModelView)swModel.ActiveView;
swModelView.RotateAboutCenter(0.00576690959137988, 0.0165762426445693);
swModelView = (ModelView)swModel.ActiveView;
swModelView.RotateAboutCenter(0.00576690959137988, 0.0414406066114233);
swModelView = (ModelView)swModel.ActiveView;
swModelView.RotateAboutCenter(0.00576690959137988, 0.0331524852891386);
swModelView = (ModelView)swModel.ActiveView;
swModelView.RotateAboutCenter(0.00576690959137988, 0.0331524852891386);
swModelView = (ModelView)swModel.ActiveView;
swModelView.RotateAboutCenter(0.0115338191827598, 0.0580168492559925);
swModelView = (ModelView)swModel.ActiveView;
swModelView.RotateAboutCenter(0.00576690959137988, 0.0414406066114233);
swModelView = (ModelView)swModel.ActiveView;
swModelView.RotateAboutCenter(0.00576690959137988, 0.0580168492559925);
swModelView = (ModelView)swModel.ActiveView;
swModelView.RotateAboutCenter(0, 0.0663049705782772);
swModelView = (ModelView)swModel.ActiveView;
swModelView.RotateAboutCenter(0.00576690959137988, 0.0663049705782772);
swModelView = (ModelView)swModel.ActiveView;
swModelView.RotateAboutCenter(0.00576690959137988, 0.1077455771897);
swModelView = (ModelView)swModel.ActiveView;
swModelView.RotateAboutCenter(0, 0.0994574558674158);
swModelView = (ModelView)swModel.ActiveView;
swModelView.RotateAboutCenter(0, 0.0745930919005618);
swModelView = (ModelView)swModel.ActiveView;
swModelView.RotateAboutCenter(0, 0.0911693345451311);
swModelView = (ModelView)swModel.ActiveView;
swModelView.RotateAboutCenter(0, 0.0828812132228465);
swModelView = (ModelView)swModel.ActiveView;
swModelView.RotateAboutCenter(0, 0.0745930919005618);
swModelView = (ModelView)swModel.ActiveView;
swModelView.RotateAboutCenter(0, 0.0580168492559925);
swModelView = (ModelView)swModel.ActiveView;
swModelView.RotateAboutCenter(0, 0.0248643639668539);
swModelView = (ModelView)swModel.ActiveView;
swModelView.RotateAboutCenter(0, 0.00828812132228465);
swModel.ClearSelection2(true);

System.Diagnostics.Debugger.Break();
//Examine Plane1
//Press F5 to continue

//Update Plane1 so that it is parallel to the screen
status = swModelDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, false, 0, null, 0);
swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
swRefPlaneFeatureData = (RefPlaneFeatureData)swFeature.GetDefinition();
swRefPlaneFeatureData.UpdatePlane = true;
swFeature.ModifyDefinition(swRefPlaneFeatureData, swModel, null);

```

```

}

```

```

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;

```

```

}
}

```

Update Plane Example (VBA)

This example shows how to update a reference plane so that it is parallel to the screen.

```
'-----  
' Preconditions: Verify that the specified part template exists.  
'-----  
' Postconditions:  
' 1. Creates Boss-Extrude1 and Plane1.  
' 2. Rotates Boss-Extrude1.  
' 3. Examine Plane1 in the graphics area.  
' 4. Press F5.  
' 5. Updates Plane1 so that it is parallel to the screen.  
' 6. Examine Plane1 in the graphics area.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSketchManager As SldWorks.SketchManager  
Dim swSketchSegment As SldWorks.SketchSegment  
Dim swFeatureManager As SldWorks.FeatureManager  
Dim swFeature As SldWorks.Feature  
Dim swSelectionMgr As SldWorks.SelectionMgr  
Dim swRefPlane As SldWorks.RefPlane  
Dim swModelView As SldWorks.ModelView  
Dim swRefPlaneFeatureData As SldWorks.RefPlaneFeatureData  
Dim status As Boolean  
  
Sub main()  
    Set swApp = Application.SldWorks  
  
    'Create Boss-Extrude1 and Plane1  
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\part.prtdot", 0, 0, 0)  
    Set swModelDocExt = swModel.Extension  
    status = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)  
    Set swSketchManager = swModel.SketchManager  
    swSketchManager.InsertSketch True  
    Set swSketchSegment = swSketchManager.CreateLine(-0.049503, 0.030205, 0#, -0.049503, 0#, 0#)  
    Set swSketchSegment = swSketchManager.CreateLine(-0.049503, 0#, 0#, 0#, 0#, 0#)  
    Set swSketchSegment = swSketchManager.CreateLine(0#, 0#, 0#, 0#, 0.019088, 0#)  
    Set swSketchSegment = swSketchManager.CreateLine(0#, 0.019088, 0#, -0.03503, 0.037127, 0#)  
    Set swSketchSegment = swSketchManager.CreateLine(-0.03503, 0.037127, 0#, -0.049503, 0.030205, 0#)  
    swSketchManager.InsertSketch True  
    Set swFeatureManager = swModel.FeatureManager  
    Set swFeature = swFeatureManager.FeatureExtrusion3(True, False, False, 0, 0, 0.04, 0.01, False, False, False,  
False, 1.74532925199433E-02, 1.74532925199433E-02, False, False, False, False, True, True, True, 0, 0, False)  
    Set swSelectionMgr = swModel.SelectionManager  
    swSelectionMgr.EnableContourSelection = False  
    status = swModelDocExt.SelectByID("", "VERTEX", -3.50298017733792E-02, 3.71273946939409E-02, 0.04, True, 0,  
Nothing, 0)  
    Set swRefPlane = swFeatureManager.InsertRefPlane(4096, 0, 0, 0, 0, 0)  
    swModel.ClearSelection2 True  
  
    'Rotate Boss-Extrude1  
    status = swModelDocExt.SelectByID("Boss-Extrude1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)  
    Set swModelView = swModel.ActiveView  
    swModelView.RotateAboutCenter 0, 8.28812132228465E-03  
    Set swModelView = swModel.ActiveView  
    swModelView.RotateAboutCenter 0, 8.28812132228465E-03  
    Set swModelView = swModel.ActiveView  
    swModelView.RotateAboutCenter 5.76690959137988E-03, 0  
    Set swModelView = swModel.ActiveView  
    swModelView.RotateAboutCenter 5.76690959137988E-03, 1.65762426445693E-02  
    Set swModelView = swModel.ActiveView  
    swModelView.RotateAboutCenter 5.76690959137988E-03, 8.28812132228465E-03  
    Set swModelView = swModel.ActiveView  
    swModelView.RotateAboutCenter 0, 1.65762426445693E-02
```

```

Set swModelView = swModel.ActiveView
swModelView.RotateAboutCenter 1.15338191827598E-02, 2.48643639668539E-02
Set swModelView = swModel.ActiveView
swModelView.RotateAboutCenter 1.15338191827598E-02, 3.31524852891386E-02
Set swModelView = swModel.ActiveView
swModelView.RotateAboutCenter 5.76690959137988E-03, 2.48643639668539E-02
Set swModelView = swModel.ActiveView
swModelView.RotateAboutCenter 5.76690959137988E-03, 1.65762426445693E-02
Set swModelView = swModel.ActiveView
swModelView.RotateAboutCenter 0, 2.48643639668539E-02
Set swModelView = swModel.ActiveView
swModelView.RotateAboutCenter 5.76690959137988E-03, 1.65762426445693E-02
Set swModelView = swModel.ActiveView
swModelView.RotateAboutCenter 5.76690959137988E-03, 1.65762426445693E-02
Set swModelView = swModel.ActiveView
swModelView.RotateAboutCenter 5.76690959137988E-03, 4.14406066114233E-02
Set swModelView = swModel.ActiveView
swModelView.RotateAboutCenter 5.76690959137988E-03, 3.31524852891386E-02
Set swModelView = swModel.ActiveView
swModelView.RotateAboutCenter 5.76690959137988E-03, 3.31524852891386E-02
Set swModelView = swModel.ActiveView
swModelView.RotateAboutCenter 1.15338191827598E-02, 5.80168492559925E-02
Set swModelView = swModel.ActiveView
swModelView.RotateAboutCenter 5.76690959137988E-03, 4.14406066114233E-02
Set swModelView = swModel.ActiveView
swModelView.RotateAboutCenter 5.76690959137988E-03, 5.80168492559925E-02
Set swModelView = swModel.ActiveView
swModelView.RotateAboutCenter 0, 6.63049705782772E-02
Set swModelView = swModel.ActiveView
swModelView.RotateAboutCenter 5.76690959137988E-03, 6.63049705782772E-02
Set swModelView = swModel.ActiveView
swModelView.RotateAboutCenter 5.76690959137988E-03, 0.1077455771897
Set swModelView = swModel.ActiveView
swModelView.RotateAboutCenter 0, 9.94574558674158E-02
Set swModelView = swModel.ActiveView
swModelView.RotateAboutCenter 0, 7.45930919005618E-02
Set swModelView = swModel.ActiveView
swModelView.RotateAboutCenter 0, 9.11693345451311E-02
Set swModelView = swModel.ActiveView
swModelView.RotateAboutCenter 0, 8.28812132228465E-02
Set swModelView = swModel.ActiveView
swModelView.RotateAboutCenter 0, 7.45930919005618E-02
Set swModelView = swModel.ActiveView
swModelView.RotateAboutCenter 0, 5.80168492559925E-02
Set swModelView = swModel.ActiveView
swModelView.RotateAboutCenter 0, 2.48643639668539E-02
Set swModelView = swModel.ActiveView
swModelView.RotateAboutCenter 0, 8.28812132228465E-03
swModel.ClearSelection2 True

Stop
'Examine Panel
'Press F5 to continue

'Update Panel so that it is parallel to the screen
status = swModelDocExt.SelectByID("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
Set swRefPlaneFeatureData = swFeature.GetDefinition
swRefPlaneFeatureData.UpdatePlane = True
swFeature.ModifyDefinition swRefPlaneFeatureData, swModel, Nothing

```

End Sub

Update Plane Example (VB.NET)

This example shows how to update a reference plane so that it is parallel to the screen.

```

'-----
' Preconditions: Verify that the specified part template exists.
'
' Postconditions:
' 1. Creates Boss-Extrude1 and Plane1.
' 2. Rotates Boss-Extrude1.
' 3. Examine Plane1 in the graphics area.
' 4. Press F5.
' 5. Updates Plane1 so that it is parallel to the screen.
' 6. Examine Plane1 in the graphics area.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

```

```
Partial Class SolidWorksMacro
```

```

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchManager As SketchManager
        Dim swSketchSegment As SketchSegment
        Dim swFeatureManager As FeatureManager
        Dim swFeature As Feature
        Dim swSelectionMgr As SelectionMgr
        Dim swRefPlane As RefPlane
        Dim swModelView As ModelView
        Dim swRefPlaneFeatureData As RefPlaneFeatureData
        Dim status As Boolean

        'Create Boss-Extrude1 and Plane1
        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\part.prt", 0, 0, 0)
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swSketchManager = swModel.SketchManager
        swSketchManager.InsertSketch(True)
        swSketchSegment = swSketchManager.CreateLine(-0.049503, 0.030205, 0.0#, -0.049503, 0.0#, 0.0#)
        swSketchSegment = swSketchManager.CreateLine(-0.049503, 0.0#, 0.0#, 0.0#, 0.0#, 0.0#)
        swSketchSegment = swSketchManager.CreateLine(0.0#, 0.0#, 0.0#, 0.0#, 0.019088, 0.0#)
        swSketchSegment = swSketchManager.CreateLine(0.0#, 0.019088, 0.0#, -0.03503, 0.037127, 0.0#)
        swSketchSegment = swSketchManager.CreateLine(-0.03503, 0.037127, 0.0#, -0.049503, 0.030205, 0.0#)
        swSketchManager.InsertSketch(True)
        swFeatureManager = swModel.FeatureManager
        swFeature = swFeatureManager.FeatureExtrusion3(True, False, False, 0, 0, 0.04, 0.01, False, False, False, F
also, 0.0174532925199433, 0.0174532925199433, False, False, False, False, True, True, True, 0, 0, False)
        swSelectionMgr = swModel.SelectionManager
        swSelectionMgr.EnableContourSelection = False
        status = swModelDocExt.SelectByID2("", "VERTEX", -0.0350298017733792, 0.0371273946939409, 0.04, True, 0, No
thing, 0)
        swRefPlane = swFeatureManager.InsertRefPlane(4096, 0, 0, 0, 0, 0)
        swModel.ClearSelection2(True)

        'Rotate Boss-Extrude1
        status = swModelDocExt.SelectByID2("Boss-Extrude1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swModelView = swModel.ActiveView
        swModelView.RotateAboutCenter(0, 0.00828812132228465)
        swModelView = swModel.ActiveView
        swModelView.RotateAboutCenter(0, 0.00828812132228465)
        swModelView = swModel.ActiveView
        swModelView.RotateAboutCenter(0.00576690959137988, 0)
        swModelView = swModel.ActiveView
        swModelView.RotateAboutCenter(0.00576690959137988, 0.0165762426445693)
        swModelView = swModel.ActiveView
        swModelView.RotateAboutCenter(0.00576690959137988, 0.00828812132228465)
        swModelView = swModel.ActiveView
        swModelView.RotateAboutCenter(0, 0.0165762426445693)
        swModelView = swModel.ActiveView
        swModelView.RotateAboutCenter(0.0115338191827598, 0.0248643639668539)
        swModelView = swModel.ActiveView
        swModelView.RotateAboutCenter(0.0115338191827598, 0.0331524852891386)
        swModelView = swModel.ActiveView
        swModelView.RotateAboutCenter(0.00576690959137988, 0.0248643639668539)
        swModelView = swModel.ActiveView
    
```

```

swModelView.RotateAboutCenter(0.00576690959137988, 0.0165762426445693)
swModelView = swModel.ActiveView
swModelView.RotateAboutCenter(0, 0.0248643639668539)
swModelView = swModel.ActiveView
swModelView.RotateAboutCenter(0.00576690959137988, 0.0165762426445693)
swModelView = swModel.ActiveView
swModelView.RotateAboutCenter(0.00576690959137988, 0.0165762426445693)
swModelView = swModel.ActiveView
swModelView.RotateAboutCenter(0.00576690959137988, 0.0414406066114233)
swModelView = swModel.ActiveView
swModelView.RotateAboutCenter(0.00576690959137988, 0.0331524852891386)
swModelView = swModel.ActiveView
swModelView.RotateAboutCenter(0.00576690959137988, 0.0331524852891386)
swModelView = swModel.ActiveView
swModelView.RotateAboutCenter(0.0115338191827598, 0.0580168492559925)
swModelView = swModel.ActiveView
swModelView.RotateAboutCenter(0.00576690959137988, 0.0414406066114233)
swModelView = swModel.ActiveView
swModelView.RotateAboutCenter(0.00576690959137988, 0.0580168492559925)
swModelView = swModel.ActiveView
swModelView.RotateAboutCenter(0, 0.0663049705782772)
swModelView = swModel.ActiveView
swModelView.RotateAboutCenter(0.00576690959137988, 0.0663049705782772)
swModelView = swModel.ActiveView
swModelView.RotateAboutCenter(0.00576690959137988, 0.1077455771897)
swModelView = swModel.ActiveView
swModelView.RotateAboutCenter(0, 0.0994574558674158)
swModelView = swModel.ActiveView
swModelView.RotateAboutCenter(0, 0.0745930919005618)
swModelView = swModel.ActiveView
swModelView.RotateAboutCenter(0, 0.0911693345451311)
swModelView = swModel.ActiveView
swModelView.RotateAboutCenter(0, 0.0828812132228465)
swModelView = swModel.ActiveView
swModelView.RotateAboutCenter(0, 0.0745930919005618)
swModelView = swModel.ActiveView
swModelView.RotateAboutCenter(0, 0.0580168492559925)
swModelView = swModel.ActiveView
swModelView.RotateAboutCenter(0, 0.0248643639668539)
swModelView = swModel.ActiveView
swModelView.RotateAboutCenter(0, 0.00828812132228465)
swModel.ClearSelection2(True)

Stop
'Examine Plane1
'Press F5 to continue

'Update Plane1 so that it is parallel to the screen
status = swModelDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
swRefPlaneFeatureData = swFeature.GetDefinition
swRefPlaneFeatureData.UpdatePlane = True
swFeature.ModifyDefinition(swRefPlaneFeatureData, swModel, Nothing)

```

End Sub

```

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Update Weldment Cut List and Fire Post-Notify Event Example (C#)

This example shows how to handle the post-notification event that fires when the weldment cut list is updated.

```
//-----  
// Preconditions:  
// Specified file to open exists.  
//  
// Postconditions:  
// 1. Right-click on the Cut list folder.  
// 2. Select Update from the right-click menu.  
// 3. A message box displays (look in the taskbar for the hidden dialog).  
// 4. Click OK.  
// 5. Stop the macro (CTRL-ALT-Break).  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
namespace UpdateWeldmentCutListandFirePostNotifyEvent_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        public PartDoc swPart;  
        public void Main()  
        {  
            swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swStopDebuggingVstaOnExit,  
false);  
            ModelDoc2 swModel = default(ModelDoc2);  
            DocumentSpecification swDocSpecification = default(DocumentSpecification);  
            swDocSpecification =  
(DocumentSpecification)swApp.GetOpenDocSpec("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS  
2018\\samples\\tutorial\\weldments\\weldment_box2.sldprt");  
            swDocSpecification.DocumentType = (int)swDocumentTypes_e.swDocPART;  
            swModel = swApp.OpenDoc7(swDocSpecification);  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            swPart = (PartDoc)swModel;  
  
            // Set up event  
            AttachEventHandlers();  
        }  
        public void AttachEventHandlers()  
        {  
            AttachSWEvents();  
        }  
    }  
}
```



```

public void AttachSWEvents()
{
    swPart.WeldmentCutListUpdatePostNotify += this.swPart_WeldmentCutListUpdatePostNotify;
}
public int swPart_WeldmentCutListUpdatePostNotify()
{
    System.Windows.Forms.MessageBox.Show("The cut list is updated.");
    swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swStopDebuggingVstaOnExit,
true);
    return 0;
}
public SldWorks swApp;
}
}

```

Update Weldment Cut List and Fire Post-Notify Event Example (VBA)

This example shows how to handle the post-notification event that fires when the weldment cut list is updated.

```
' -----  
' Preconditions:  
' 1. Create main module with Main module code.  
' 2. Insert Class1 module with Class1 module code.  
' 3. Specified file to open exists.  
' 4. Run macro (F5).  
'  
' Postconditions:  
' 1. Right-click on the Cut list folder.  
' 2. Select Update from the right-click menu.  
' 3. A message box displays.  
' 4. Click OK.  
' 5. Stop the macro.  
' -----  
' Main module  
' -----  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swDocSpecification As SldWorks.DocumentSpecification  
Dim swPartEvents As Class1  
Dim swPart As SldWorks.PartDoc  
Option Explicit  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swDocSpecification = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\weldments\weldment_box2.sldprt")  
    swDocSpecification.InteractiveComponentSelection = True  
    swDocSpecification.DocumentType = swDocPART  
    Set swModel = swApp.OpenDoc7(swDocSpecification)  
  
    Set swModel = swApp.ActiveDoc  
    Set swPart = swModel  
  
    ' Set up event  
  
    Set swPartEvents = New Class1  
    Set swPartEvents.swPart = swPart  
End Sub
```

```
' .....  
' Class1 module  
' .....  
  
Public WithEvents swPart As SldWorks.PartDoc  
Private Function swPart_WeldmentCutListUpdatePostNotify() As Long  
    MsgBox "The cut list is updated."  
End Function
```

Update Weldment Cut List and Fire Post-Notify Event Example (VB.NET)

This example shows how to handle the post-notification event that fires when the weldment cut list is updated.

```
' -----  
' Preconditions: Specified file to open exists.  
'  
' Postconditions:  
' 1. Right-click on the Cut list folder.  
' 2. Select Update from the right-click menu.  
' 3. A message box displays (look in the taskbar for the hidden dialog).  
' 4. Click OK.  
' 5. Stop the macro.  
' -----  
  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System  
Partial Class SolidWorksMacro  
    Public WithEvents swPart As PartDoc  
    Public Sub main()  
        swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swStopDebuggingVstaOnExit, False)  
        Dim swModel As ModelDoc2  
        Dim swDocSpecification As DocumentSpecification  
        swDocSpecification = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\weldments\weldment_box2.sldprt")  
        swDocSpecification.DocumentType = swDocumentTypes_e.swDocPART  
        swModel = swApp.OpenDoc7(swDocSpecification)  
        swModel = swApp.ActiveDoc  
        swPart = swModel  
        ' Set up event  
        AttachEventHandlers()  
    End Sub  
    Sub AttachEventHandlers()  
        AttachSWEvents()  
    End Sub  
    Sub AttachSWEvents()  
        AddHandler swPart.WeldmentCutListUpdatePostNotify, AddressOf  
Me.swPart_WeldmentCutListUpdatePostNotify  
    End Sub  
    Public Function swPart_WeldmentCutListUpdatePostNotify() As Integer  
        MsgBox("The cut list is updated.")  
        swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swStopDebuggingVstaOnExit, True)  
    End Function  
    Public swApp As SldWorks
```

End Class

Use Advanced Component Selection Example (C#)

This example shows how to:

- load an advanced selection component query file.
- select, add, delete, and save a selection criteria.

```
//-----  
// Preconditions:  
// 1. An assembly document is active.  
// 2. A selection criteria query file named c:\temp\QueryMassLess.02.sqy  
//    exists and contains one selection criteria.  
// 3. Open the Immediate window.  
//  
// Postconditions:  
// 1. Adds a selection criteria to c:\temp\QueryMassLess.02.sqy.  
// 2. Deletes first selection criteria from c:\temp\QueryMassLess.02.sqy.  
// 3. Saves selection criteria file c:\temp\QueryNewCriteria.sqy.  
// 4. Examine the Immediate window and c:\temp.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
  
namespace AdvancedSelectionCSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
  
        public void Main()  
        {  
            ModelDoc2 ModelDoc2 = default(ModelDoc2);  
            ModelDoc2 = (ModelDoc2)swApp.ActiveDoc;  
  
            int DocType = 0;  
            DocType = ModelDoc2.GetType();  
  
            if (DocType != (int)swDocumentTypes_e.swDocASSEMBLY)  
            {  
                Debug.Print("An assembly document is not active");  
                return;  
            }  
  
            ModelDoc2.ClearSelection2(true);  
  
            AssemblyDoc AssemblyDoc = default(AssemblyDoc);  
            AssemblyDoc = (AssemblyDoc)ModelDoc2;  
  
            AdvancedSelectionCriteria AdvancedSelectionCriteria = default(AdvancedSelectionCriteria);  
            // Get advanced component selection  
            AdvancedSelectionCriteria = (AdvancedSelectionCriteria)AssemblyDoc.GetAdvancedSelection();  
  
            int Count = 0;  
            // Get number of advanced component selections before loading a query  
            // Should be 0  
            Count = AdvancedSelectionCriteria.GetItemCount();  
            Debug.Print("Before loading a query, GetItemCount returned " + Count);  
        }  
    }  
}
```

```

        string CriteriaFileName = null;
        // Query file
        CriteriaFileName = "c:\\temp\\QueryMassLess.02.sqy";

        bool LoadSuccess = false;
        // Load query file
        LoadSuccess = AdvancedSelectionCriteria.LoadCriteria(CriteriaFileName);
        Debug.Print("Query " + CriteriaFileName + " load was" + (LoadSuccess == false
? " NOT " : " ") + "successful");
        ReportAllValues(AdvancedSelectionCriteria);
        bool SelectSuccess = false;
        // Select selection criteria from query list
        SelectSuccess = AdvancedSelectionCriteria.Select();
        Debug.Print("Select was" + (SelectSuccess == false ? " NOT " : " ") + "succes
sful");

        int AddRetVal = 0;
        // Add selection criteria to query list
        AddRetVal = AdvancedSelectionCriteria.AddItem("Document name --
SW Special", 0x4, "Gear", false);
        Debug.Print("AddItem returned " + AddRetVal);

        // Print values of advanced component selection criteria
        ReportAllValues(AdvancedSelectionCriteria);

        // Select first advanced component selection criteria
        SelectSuccess = AdvancedSelectionCriteria.Select();
        Debug.Print("Select was" + (SelectSuccess == false ? " NOT " : " ") + "succes
sful");

        bool DeleteStatus = false;
        // Delete first component selection criteria
        DeleteStatus = AdvancedSelectionCriteria.DeleteItem(0);
        Debug.Print("DeleteItem was" + (DeleteStatus == false ? " NOT " : " ") + "suc
cessful");

        ReportAllValues(AdvancedSelectionCriteria);
        bool SaveStatus = false;
        // Save query file
        SaveStatus = AdvancedSelectionCriteria.SaveCriteria("C:\\temp\\QueryNewCriter
ia.sqy");
        //If wanted, need to specify extension
        Debug.Print("SaveCriteria was" + (SaveStatus == false ? " NOT " : " ") + "suc
cessful");

        // Load query file that was just saved
        LoadSuccess = AdvancedSelectionCriteria.LoadCriteria("C:\\temp\\QueryNewCrite
ria.sqy");
        Debug.Print("Query " + "c:\\temp\\QueryNewCriteria.sqy" + " load was" + (Load
Success == false ? " NOT " : " ") + "successful");
        // Print contents query file
        ReportAllValues(AdvancedSelectionCriteria);
    }

    public string GetStringFromEnum(int EnumVal)
    {
        string functionReturnValue = null;
        //From enum swAdvSelectType_e
        if (EnumVal == 1)
        {
            functionReturnValue = "And";
        }
        else if (EnumVal == 2)
        {
            functionReturnValue = "Or";
        }
        else if (EnumVal == 16384)
        {

```

```

        functionReturnValue = "is yes";
    }
    else if (EnumVal == 32768)
    {
        functionReturnValue = "is no";
    }
    else if (EnumVal == 8)
    {
        functionReturnValue = "is not";
    }
    else if (EnumVal == 16)
    {
        functionReturnValue = "contains";
    }
    else if (EnumVal == 32)
    {
        functionReturnValue = "Is_Contained_By";
    }
    else if (EnumVal == 64)
    {
        functionReturnValue = "Interferes_With";
    }
    else if (EnumVal == 128)
    {
        functionReturnValue = "Does_Not_Interferes_With";
    }
    else if (EnumVal == 4)
    {
        functionReturnValue = "is (exactly)";
    }
    else if (EnumVal == 8192)
    {
        functionReturnValue = "not =";
    }
    else if (EnumVal == 512)
    {
        functionReturnValue = "<";
    }
    else if (EnumVal == 2048)
    {
        functionReturnValue = "<=";
    }
    else if (EnumVal == 4096)
    {
        functionReturnValue = "=";
    }
    else if (EnumVal == 1024)
    {
        functionReturnValue = ">=";
    }
    else if (EnumVal == 256)
    {
        functionReturnValue = ">";
    }
    else
    {
        functionReturnValue = "Condition NOT found";
    }
    return functionReturnValue;
}
public void ReportAllValues(AdvancedSelectionCriteria AdvancedSelectionCriteria)
{
    Debug.Print("");

    int Count = 0;
    Count = AdvancedSelectionCriteria.GetItemCount();
    Debug.Print("GetItemCount returned " + Count);
}

```



```

int i = 0;
string aProperty = "";
int Condition = 0;
string Value = "";
bool IsAnd = false;
int Rindex = 0;
string ConditionString = null;
string PrintString = null;

string IndexFmt = null;
string RindexFmt = null;
string AndOrFmt = null;
string PropertyFmt = null;
string ConditionFmt = null;
string ValueFmt = null;
IndexFmt = "!@!!!!!!!!!!";
RindexFmt = "!@!!!!!!!!!!!!!!";
AndOrFmt = "!@!!!!!!!!!!!!!!";
PropertyFmt = "!@!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!";
ConditionFmt = "!@!!!!!!!!!!!!!!!!!!!!!!";
ValueFmt = "#.00";

//Debug.Print
PrintString = string.Format("Index", IndexFmt) + " " + string.Format("Rindex", RindexFmt) + " " + string.Format("And/Or", AndOrFmt) + " " + string.Format("Property", PropertyFmt) + " " + string.Format("Condition", ConditionFmt) + " " + string.Format("Value", ValueFmt);
Debug.Print(PrintString);
for (i = 0; i <= Count - 1; i++)
{
    Rindex = AdvancedSelectionCriteria.GetItem(i, out aProperty, out ConditionString, out Value, out IsAnd);
    ConditionString = GetStringFromEnum(Condition);
    PrintString = string.Format(i.ToString(), IndexFmt) + " " + string.Format(Rindex.ToString(), RindexFmt) + " " + string.Format((IsAnd == false ? "OR" : "AND"), AndOrFmt) + " " + string.Format(aProperty, PropertyFmt) + " " + string.Format(ConditionString, ConditionFmt) + " " + string.Format(Value, ValueFmt);
    Debug.Print(PrintString);
}
Debug.Print("");
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}

```

Use Advanced Component Selection Example (VBA)

This example shows how to:

- load an advanced selection component query file.
- select, add, delete, and save a selection criteria.

' Preconditions:

- ' 1. An assembly document is active.
- ' 2. A selection criteria query file named **c:\templQueryMassLess.02.sqy** exists and contains one selection criteria.
- ' 3. Open the Immediate window.

' Postconditions:

- ' 1. Adds a selection criteria to **c:\templQueryMassLess.02.sqy**.
 - ' 2. Deletes first selection criteria from **c:\templQueryMassLess.02.sqy**.
 - ' 3. Saves selection criteria file **c:\templQueryNewCriteria.sqy**.
 - ' 4. Examine the Immediate window and **c:\temp**.
-

Option Explicit

Sub main()

```
Dim swAppSldWorks As SldWorks.SldWorks
Set swAppSldWorks = Application.SldWorks
```

```
Dim ModelDoc2 As SldWorks.ModelDoc2
Set ModelDoc2 = swAppSldWorks.ActiveDoc
```

```
Dim DocType As Long
DocType = ModelDoc2.GetType
```

```
If DocType <> swDocASSEMBLY Then
    Debug.Print "An assembly document is not active"
    Exit Sub
End If
```

```
ModelDoc2.ClearSelection2 True
```

```
Dim AssemblyDoc As SldWorks.AssemblyDoc
Set AssemblyDoc = ModelDoc2
```

```
Dim AdvancedSelectionCriteria As SldWorks.AdvancedSelectionCriteria
' Get advanced component selection
Set AdvancedSelectionCriteria = AssemblyDoc.GetAdvancedSelection
```

```
Dim Count As Long
' Get number of advanced component selections before loading a query
' Should be 0
Count = AdvancedSelectionCriteria.GetItemCount
Debug.Print "Before loading a query, GetItemCount returned " & Count
```

```
Dim CriteriaFileName As String
' Query file
CriteriaFileName = "c:\temp\QueryMassLess.02.sqy"
```

```
Dim LoadSuccess As Boolean
```

```

' Load query file
LoadSuccess = AdvancedSelectionCriteria.LoadCriteria(CriteriaFileName)
Debug.Print "Query " & CriteriaFileName & " load was" & If(LoadSuccess = False, " NOT ", " ") & "successful"
ReportAllValues AdvancedSelectionCriteria

Dim SelectSuccess As Boolean
' Select selection criteria from query list
SelectSuccess = AdvancedSelectionCriteria.Select
Debug.Print "Select was" & If(SelectSuccess = False, " NOT ", " ") & "successful"

Dim AddRetVal As Long
' Add selection criteria to query list
AddRetVal = AdvancedSelectionCriteria.AddItem("Document name -- SW Special", &H4, "Gear", False)
Debug.Print "AddItem returned " & AddRetVal

' Print values of advanced component selection criteria
ReportAllValues AdvancedSelectionCriteria

' Select first advanced component selection criteria
SelectSuccess = AdvancedSelectionCriteria.Select
Debug.Print "Select was" & If(SelectSuccess = False, " NOT ", " ") & "successful"
Dim DeleteStatus As Boolean
' Delete first component selection criteria
DeleteStatus = AdvancedSelectionCriteria.Deleteltem(0)
Debug.Print "Deleteltem was" & If(DeleteStatus = False, " NOT ", " ") & "successful"

ReportAllValues AdvancedSelectionCriteria
Dim SaveStatus As Boolean
' Save query file
SaveStatus = AdvancedSelectionCriteria.SaveCriteria("C:\temp\QueryNewCriteria.sqy") 'If wanted, need to specify extension
Debug.Print "SaveCriteria was" & If(SaveStatus = False, " NOT ", " ") & "successful"

' Load query file that was just saved
LoadSuccess = AdvancedSelectionCriteria.LoadCriteria("C:\temp\QueryNewCriteria.sqy")
Debug.Print "Query " & "c:\temp\QueryNewCriteria.sqy" & " load was" & If(LoadSuccess = False, " NOT ", " ") & "successful"
' Print contents query file
ReportAllValues AdvancedSelectionCriteria
End Sub

Function GetStringFromEnum(EnumVal As Long) As String
'From enum swAdvSelectType_e
If EnumVal = &H1 Then
    GetStringFromEnum = "And"
ElseIf EnumVal = &H2 Then
    GetStringFromEnum = "Or"
ElseIf EnumVal = &H4000 Then
    GetStringFromEnum = "is yes"
ElseIf EnumVal = &H8000 Then
    GetStringFromEnum = "is no"
ElseIf EnumVal = &H8 Then
    GetStringFromEnum = "is not"
ElseIf EnumVal = &H10 Then
    GetStringFromEnum = "contains"
ElseIf EnumVal = &H20 Then
    GetStringFromEnum = "Is_Contained_By"
ElseIf EnumVal = &H40 Then

```

```

        GetStringFromEnum = "Interferes_With"
Elseif EnumVal = &H80 Then
    GetStringFromEnum = "Does_Not_Interferes_With"
Elseif EnumVal = &H4 Then
    GetStringFromEnum = "is (exactly)"
Elseif EnumVal = &H2000 Then
    GetStringFromEnum = "not ="
Elseif EnumVal = &H200 Then
    GetStringFromEnum = "<"
Elseif EnumVal = &H800 Then
    GetStringFromEnum = "<="
Elseif EnumVal = &H1000 Then
    GetStringFromEnum = "="
Elseif EnumVal = &H400 Then
    GetStringFromEnum = ">="
Elseif EnumVal = &H100 Then
    GetStringFromEnum = ">"
Else
    GetStringFromEnum = "Condition NOT found"
End If
End Function

Sub ReportAllValues(AdvancedSelectionCriteria As SldWorks.AdvancedSelectionCriteria)
    Debug.Print

    Dim Count As Long
    Count = AdvancedSelectionCriteria.GetItemCount
    Debug.Print "GetItemCount returned " & Count

    Dim i As Long
    Dim Property As String
    Dim Condition As Long
    Dim Value As String
    Dim IsAnd As Boolean
    Dim Rindex As Long
    Dim ConditionString As String
    Dim PrintString As String

    Dim IndexFmt As String
    Dim RindexFmt As String
    Dim AndOrFmt As String
    Dim PropertyFmt As String
    Dim ConditionFmt As String
    Dim ValueFmt As String
    IndexFmt = "!@@@@@@@@@"
    RindexFmt = "!@@@@@@@@@"
    AndOrFmt = "!@@@@@@@@@"
    PropertyFmt = "!@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@"
    ConditionFmt = "!@@@@@@@@@@@@@@@@@@@@@"
    ValueFmt = "#.00"

    'Debug.Print

    PrintString = Format("Index", IndexFmt) & Format("Rindex", RindexFmt) & Format("And/Or", AndOrFmt) & Format("Property", PropertyFmt) & Format("Condition",
    ConditionFmt) & Format("Value", ValueFmt)
    Debug.Print PrintString

```

```
For i = 0 To Count - 1
    Rindex = AdvancedSelectionCriteria.GetItem(i, Property, Condition, Value, IsAnd)
    ConditionString = GetStringFromEnum(Condition)
    PrintString = Format(i, IndexFmt) & Format(Rindex, RindexFmt) & Format(If(IsAnd = False, "OR", "AND"), AndOrFmt) & Format(Property, PropertyFmt) &
    Format(ConditionString, ConditionFmt) & Format(Value, ValueFmt)
    Debug.Print PrintString
Next i
Debug.Print
End Sub
```

Use Advanced Component Selection Example (VB.NET)

This example shows how to:

- load an advanced selection component query file.
- select, add, delete, and save a selection criteria.

```
'-----  
' Preconditions:  
' 1. An assembly document is active.  
' 2. A selection criteria query file named c:\temp\QueryMassLess.02.sqy  
'    exists and contains one selection criteria.  
' 3. Open the Immediate window.  
'-----  
' Postconditions:  
' 1. Adds a selection criteria to c:\temp\QueryMassLess.02.sqy.  
' 2. Deletes first selection criteria from c:\temp\QueryMassLess.02.sqy.  
' 3. Saves selection criteria file c:\temp\QueryNewCriteria.sqy.  
' 4. Examine the Immediate window and c:\temp.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics
```

Partial Class SolidWorksMacro

```
Public Sub main()  
    Dim ModelDoc2 As ModelDoc2  
    ModelDoc2 = swApp.ActiveDoc  
  
    Dim DocType As Integer  
    DocType = ModelDoc2.GetType  
  
    If DocType <> swDocumentTypes_e.swDocASSEMBLY Then  
        Debug.Print("An assembly document is not active")  
        Exit Sub  
    End If  
  
    ModelDoc2.ClearSelection2(True)  
  
    Dim AssemblyDoc As AssemblyDoc  
    AssemblyDoc = ModelDoc2  
  
    Dim AdvancedSelectionCriteria As AdvancedSelectionCriteria  
    ' Get advanced component selection  
    AdvancedSelectionCriteria = AssemblyDoc.GetAdvancedSelection  
  
    Dim Count As Integer  
    ' Get number of advanced component selections before loading a query  
    ' Should be 0  
    Count = AdvancedSelectionCriteria.GetItemCount  
    Debug.Print("Before loading a query, GetItemCount returned " & Count)  
  
    Dim CriteriaFileName As String  
    ' Query file  
    CriteriaFileName = "c:\temp\QueryMassLess.02.sqy"  
  
    Dim LoadSuccess As Boolean  
    ' Load query file
```

```

        LoadSuccess = AdvancedSelectionCriteria.LoadCriteria(CriteriaFileName)
        Debug.Print("Query " & CriteriaFileName & " load was" & IIf(LoadSuccess = False,
" NOT ", " ") & "successful")
        ReportAllValues(AdvancedSelectionCriteria)
        Dim SelectSuccess As Boolean
        ' Select selection criteria from query list
        SelectSuccess = AdvancedSelectionCriteria.Select
        Debug.Print("Select was" & IIf(SelectSuccess = False, " NOT ", " ") & "successful")
    ")

    Dim AddRetVal As Integer
    ' Add selection criteria to query list
    AddRetVal = AdvancedSelectionCriteria.AddItem("Document name --
SW Special", &H4, "Gear", False)
    Debug.Print("AddItem returned " & AddRetVal)

    ' Print values of advanced component selection criteria
    ReportAllValues(AdvancedSelectionCriteria)

    ' Select first advanced component selection criteria
    SelectSuccess = AdvancedSelectionCriteria.Select
    Debug.Print("Select was" & IIf(SelectSuccess = False, " NOT ", " ") & "successful")
")

    Dim DeleteStatus As Boolean
    ' Delete first component selection criteria
    DeleteStatus = AdvancedSelectionCriteria.DeleteItem(0)
    Debug.Print("DeleteItem was" & IIf(DeleteStatus = False, " NOT ", " ") & "success
ful")

    ReportAllValues(AdvancedSelectionCriteria)
    Dim SaveStatus As Boolean
    ' Save query file
    SaveStatus = AdvancedSelectionCriteria.SaveCriteria("C:\temp\QueryNewCriteria.sqy")
    'If wanted, need to specify extension
    Debug.Print("SaveCriteria was" & IIf(SaveStatus = False, " NOT ", " ") & "success
ful")

    ' Load query file that was just saved
    LoadSuccess = AdvancedSelectionCriteria.LoadCriteria("C:\temp\QueryNewCriteria.sq
y")
    Debug.Print("Query " & "c:\temp\QueryNewCriteria.sqy" & " load was" & IIf(LoadSuc
cess = False, " NOT ", " ") & "successful")
    ' Print contents query file
    ReportAllValues(AdvancedSelectionCriteria)
End Sub

Function GetStringFromEnum(ByVal EnumVal As Integer) As String
    'From enum swAdvSelcType_e
    If EnumVal = 1 Then
        GetStringFromEnum = "And"
    ElseIf EnumVal = 2 Then
        GetStringFromEnum = "Or"
    ElseIf EnumVal = 16384 Then
        GetStringFromEnum = "is yes"
    ElseIf EnumVal = 32768 Then
        GetStringFromEnum = "is no"
    ElseIf EnumVal = 8 Then
        GetStringFromEnum = "is not"
    ElseIf EnumVal = 16 Then
        GetStringFromEnum = "contains"
    ElseIf EnumVal = 32 Then
        GetStringFromEnum = "Is_Contained_By"
    ElseIf EnumVal = 64 Then
        GetStringFromEnum = "Interferes_With"
    ElseIf EnumVal = 128 Then
        GetStringFromEnum = "Does_Not_Interferes_With"
    ElseIf EnumVal = 4 Then

```

```

        GetStringFromEnum = "is (exactly)"
    ElseIf EnumVal = 8192 Then
        GetStringFromEnum = "not ="
    ElseIf EnumVal = 512 Then
        GetStringFromEnum = "<"
    ElseIf EnumVal = 2048 Then
        GetStringFromEnum = "<="
    ElseIf EnumVal = 4096 Then
        GetStringFromEnum = "="
    ElseIf EnumVal = 1024 Then
        GetStringFromEnum = ">="
    ElseIf EnumVal = 256 Then
        GetStringFromEnum = ">"
    Else
        GetStringFromEnum = "Condition NOT found"
    End If
End Function
Sub ReportAllValues(ByVal AdvancedSelectionCriteria As AdvancedSelectionCriteria)
    Debug.Print("")

    Dim Count As Integer
    Count = AdvancedSelectionCriteria.GetItemCount
    Debug.Print("GetItemCount returned " & Count)

    Dim i As Integer
    Dim aProperty As String = ""
    Dim Condition As Integer
    Dim Value As String = ""
    Dim IsAnd As Boolean
    Dim Rindex As Integer
    Dim ConditionString As String
    Dim PrintString As String

    Dim IndexFmt As String
    Dim RindexFmt As String
    Dim AndOrFmt As String
    Dim PropertyFmt As String
    Dim ConditionFmt As String
    Dim ValueFmt As String
    IndexFmt = "!@@@@@@@@@@"
    RindexFmt = "!@@@@@@@@@@"
    AndOrFmt = "!@@@@@@@@@@"
    PropertyFmt = "!@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@"
    ConditionFmt = "!@@@@@@@@@@@@@@@@@@@@@"
    ValueFmt = "#.00"

    'Debug.Print
    PrintString = String.Format("Index", IndexFmt) & " " & String.Format("Rindex",
, RindexFmt) & " " & String.Format("And/Or", AndOrFmt) & " " & String.Format("Property"
, PropertyFmt) & " " & String.Format("Condition", ConditionFmt) & "
" & String.Format("Value", ValueFmt)
    Debug.Print(PrintString)
    For i = 0 To Count - 1
        Rindex = AdvancedSelectionCriteria.GetItem(i, aProperty, Condition, Value, Is
And)
        ConditionString = GetStringFromEnum(Condition)
        PrintString = String.Format(i, IndexFmt) & " " & String.Format(Rindex
, RindexFmt) & " " & String.Format(If(IsAnd = False, "OR", "AND"), AndOrFmt) & "
" & String.Format(aProperty, PropertyFmt) & " " & String.Format(ConditionString, Con
ditionFmt) & " " & String.Format(Value, ValueFmt)
        Debug.Print(PrintString)
    Next i
    Debug.Print("")

End Sub

''' <summary>

```



```
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

End Class

Use Hole Wizard Feature Data Object to Create Hole Wizard Feature Example (VBA)

This example shows how to use an existing hole wizard feature data object to create another hole wizard feature.

```
'-----
' Preconditions: Verify that the specified part to open exists.
'
' Postconditions:
' 1. Opens the part.
' 2. Creates a hole wizard feature.
' 3. Creates another hole wizard feature
'    based on the existing hole wizard feature
'    and selected entities.
' 4. Examine the graphics area and FeatureManager design tree.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swFeat As SldWorks.Feature
Dim swWzdHole As SldWorks.WizardHoleFeatureData2
Dim swFeatMgr As SldWorks.FeatureManager
Dim boolstatus As Boolean
Dim longstatus As Long
Dim longwarnings As Long
Dim HoleType As Long
Dim StandardIndex As Long
Dim FastenerTypeIndex As Long
Dim SSize As String
Dim EndType As Long
Dim ConvFactorLength As Double
Dim ConvFactorAngle As Double
Dim Diameter As Double
Dim Depth As Double
Dim Length As Double
Dim ScrewFit As Double
Dim DrillAngle As Double
Dim NearCsinkDiameter As Double
Dim NearCsinkAngle As Double
Dim FarCsinkDiameter As Double
Dim FarCsinkAngle As Double
Dim Offset As Double
Dim ThreadClass As String
Dim CounterBoreDiameter As Double
Dim CounterBoreDepth As Double
Dim HeadClearance As Double
Dim BotCsinkDiameter As Double
Dim BotCsinkAngle As Double
```

Sub main()

```
    Set swApp = Application.SldWorks
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\api\block20.sldprt", 1, 0, "", longstatus, longwarnings)
    Set swModelDocExt = swModel.Extension
    Set swFeatMgr = swModel.FeatureManager
    Set swSelMgr = swModel.SelectionManager

    ' Create hole wizard feature
    boolstatus = swModelDocExt.SelectByID2("", "FACE", 2.21497493083689E-03, 3.96239999998897E-02,
-4.21939153824269E-03, False, 0, Nothing, 0)
    HoleType = swWzdGeneralHoleTypes_e.swWzdCounterBore
    StandardIndex = swWzdHoleStandards_e.swStandardAnsiInch
    FastenerTypeIndex = swWzdHoleStandardFastenerTypes_e.swStandardAnsiInchBinding
    SSize = "#12"
    EndType = swEndConditions_e.swEndCondThroughAll
```

```

ConvFactorLength = 25.4 / 1000      'Convert inches to meters
ConvFactorAngle = (22 / 7) / 180    'Convert degrees to radians
Diameter = 0.5 * ConvFactorLength
Depth = -1
Length = -1
' Value1 to Value12 arguments; SOLIDWORKS Design
' ignores parameters set to -1
CounterBoreDiameter = 0.6 * ConvFactorLength      'Value1
CounterBoreDepth = 0.2 * ConvFactorLength          'Value2
HeadClearance = -1                                'Value3
ScrewFit = -1                                      'Value4
DrillAngle = -1                                    'Value5
NearCsinkDiameter = -1                             'Value6
NearCsinkAngle = -1                                'Value7
BotCsinkDiameter = -1                              'Value8
BotCsinkAngle = -1                                 'Value9
FarCsinkDiameter = -1                              'Value10
FarCsinkAngle = -1                                 'Value11
Offset = -1                                         'Value12
ThreadClass = ""
Set swFeat = swFeatMgr.HoleWizard5(HoleType, StandardIndex, FastenerTypeIndex, SSize, EndType, Diameter,
Depth, Length, CounterBoreDiameter, CounterBoreDepth, HeadClearance, ScrewFit, DrillAngle, NearCsinkDiameter,
NearCsinkAngle, BotCsinkDiameter, BotCsinkAngle, FarCsinkDiameter, FarCsinkAngle, Offset, ThreadClass, False,
False, False, False, False, False)

' Get the hole wizard feature data object
boolstatus = swModelDocExt.SelectByID2("CBORE for #12 Binding Head Machine Screw1", "BODYFEATURE", 0, 0, 0,
False, 0, Nothing, 0)
Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
Set swWzdHole = swFeat.GetDefinition
swModel.ClearSelection2 True

Set swFeat = Nothing

' Select the reference entities to use to create another hole wizard feature
boolstatus = swModelDocExt.SelectByID2("", "FACE", 4.31776836886115E-02, 3.96239999998897E-02,
-4.15372044423066E-02, False, 0, Nothing, 0)
boolstatus = swModelDocExt.SelectByID2("", "VERTEX", 6.46002912861796E-02, 0.039624, -0.063937043942379, True,
2, Nothing, 0)

' Create the new hole wizard feature
Set swFeat = swFeatMgr.CreateFeature(swWzdHole)
End Sub

```

Use If Function When Adding an Equation Example (VBA)

This example shows how to use the Visual Basic If function when adding an equation.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\cover_with_dimensions.sldprt.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Adds an equation using the Visual Basic If function.  
' 2. Examine the Immediate window.  
'  
' NOTE: Because the part is used elsewhere, do not save changes.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swEqnMgr As SldWorks.EquationMgr  
    Dim i As Long  
    Dim nCount As Long  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swEqnMgr = swModel.GetEquationMgr  
    Debug.Print "File = " & swModel.GetPathName  
    nCount = swEqnMgr.GetCount  
  
    ' List the existing equations and their calculated values  
    For i = 0 To nCount - 1  
        Debug.Print "    Eqn(" & i & ") = " & swEqnMgr.Equation(i)  
        Debug.Print "        Value = " & swEqnMgr.Value(i)  
    Next i  
  
    ' Add and solve an equation that uses the Visual Basic If function  
    ' Delay evaluating the equation by passing false into the third  
    ' argument of Add2  
    swEqnMgr.Add2 i, ("TXD2@Scheme15" = (If("TXD1@Scheme15">20, 25, 20))+3), False  
    Debug.Print "    Eqn(" & i & ") = " & swEqnMgr.Equation(i)  
  
    ' Evaluate all equations  
    Dim ret As Integer  
    ret = swEqnMgr.EvaluateAll  
    Debug.Print "        Value = " & swEqnMgr.Value(i)  
    Debug.Print "        Index of equation = " & swEqnMgr.Status  
  
End Sub
```

Use Persistent Reference Example (C#)

Persistent references are a means of returning to an item between sessions of SOLIDWORKS Design. Persistent references are similar to attributes, but are easier to use. In almost all cases, persistent references can, and should, be used in place of attributes.

This example:

- shows how to use persistent references.
- is primarily diagnostic code that determines whether the currently selected item has a persistent reference from which an object can be retrieved.

```
//-----  
// Preconditions:  
// 1. Open a part, assembly, or drawing.  
// 2. Select an entity.  
//  
// Postconditions:  
// 1. Gets the selected entity type and its PID.  
// 2. Examine the Immediate window.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
namespace UsePersistentReference_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
  
        public string GetStringFromID(SldWorks swApp, ModelDoc2 swModel, byte[] vPIDarr)  
        {  
            string functionReturnValue = null;  
  
            foreach (int vPID in vPIDarr) {  
                functionReturnValue = functionReturnValue + vPID.ToString("###000");  
            }  
            return functionReturnValue;  
        }  
  
        public object GetObjectFromString(SldWorks swApp, ModelDoc2 swModel, string IDstring)  
        {  
            object functionReturnValue = null;  
            ModelDocExtension swModExt = default(ModelDocExtension);  
            byte[] ByteStream = new byte[IDstring.Length/3];  
            object vPIDarr = null;  
            int nRetVal = 0;  
            int i = 0;  
  
            swModExt = swModel.Extension;  
            for (i = 0; i <= IDstring.Length - 3; i += 3)  
            {  
                int j;  
                j = i / 3;  
                ByteStream[j] = Convert.ToByte(IDstring.Substring(i, 3));  
            }  
  
            vPIDarr = ByteStream;  
  
            functionReturnValue = swModExt.GetObjectByPersistReference3((vPIDarr), out nRetVal);  
  
            Debug.Assert((int)swPersistReferencedObjectStates_e.swPersistReferencedObject_0k ==  
nRetVal);  
  
            Debug.Assert((functionReturnValue != null));  
            return functionReturnValue;  
        }  
    }  
}
```

```
}
```

```
public void Main()
```

```
{
```

```
    ModelDoc2 swModel = default(ModelDoc2);
```

```
    ModelDocExtension swModExt = default(ModelDocExtension);
```

```
    SelectionMgr swSelMgr = default(SelectionMgr);
```

```
    object swSelObj = null;
```

```
    object swObj = null;
```

```
    byte[] vPIDarr = null;
```

```
    string sIDstring = null;
```

```
    swModel = (ModelDoc2)swApp.ActiveDoc;
```

```
    swModExt = swModel.Extension;
```

```
    swSelMgr = (SelectionMgr)swModel.SelectionManager;
```

```
    swSelObj = swSelMgr.GetSelectedObject6(1, -1);
```

```
    Debug.Assert((swSelObj != null));
```

```
    vPIDarr = (byte[])swModExt.GetPersistReference3(swSelObj);
```

```
    Debug.Assert((vPIDarr != null));
```

```
    Debug.Print("SelType = " + swSelMgr.GetSelectedObjectType3(1, -1));
```

```
    if ((vPIDarr == null))
```

```
    {
```

```
        Debug.Print("    ModelDocExtension::GetPersistReference3 broken");
```

```
        return;
```

```
    }
```

```
    else
```

```
    {
```

```
        Debug.Print("    ModelDocExtension::GetPersistReference3 exists");
```

```
    }
```

```
    sIDstring = GetStringFromID(swApp, swModel, vPIDarr);
```

```
    Debug.Print("    Persist Ref = " + sIDstring);
```

```
    swObj = GetObjectFromString(swApp, swModel, sIDstring);
```

```
    if ((null != swObj))
```

```
    {
```

```
        Debug.Assert(object.ReferenceEquals(swSelObj, swObj));
```

```
        Debug.Print("    ModelDocExtension::GetObjectByPersistReference3 exists");
```

```
    }
```

```
    else
```

```
    {
```

```
        Debug.Print("    ModelDocExtension::GetObjectByPersistReference3 broken");
```

```
    }
```

```
}
```

```
public SldWorks swApp;
```

```
}
```

```
}
```

Use Persistent Reference Example (VBA)

Persistent references are a means of returning to an item between sessions of SOLIDWORKS Design. Persistent references are similar to attributes, but are easier to use. In almost all cases, persistent references can, and should, be used in place of attributes.

This example:

- shows how to use persistent references.
- is primarily diagnostic code that determines whether the currently selected item has a persistent reference from which an object can be retrieved.

```
'-----  
' Preconditions:  
' 1. Open a part, assembly, or drawing.  
' 2. Select an entity.  
' 3. Open the Immediate window.  
'-----  
' Postconditions:  
' 1. Gets the selected entity type and its PID.  
' 2. Examine the Immediate window.  
'-----
```

Option Explicit

Function GetStringFromID(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, vPIDarr As Variant) As String

```
    Dim vPID As Variant  
    For Each vPID In vPIDarr  
        Debug.Assert vbByte = VarType(vPID)  
        GetStringFromID = GetStringFromID & Format(vPID, "###000")  
    Next vPID
```

End Function

Function GetObjectFromString(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, IDstring As String) As Object

```
    Dim swModExt As SldWorks.ModelDocExtension  
    Dim ByteStream() As Byte  
    Dim vPIDarr As Variant  
    Dim nRetVal As Long  
    Dim i As Long  
  
    Set swModExt = swModel.Extension  
    ReDim ByteStream(Len(IDstring) / 3 - 1)  
    For i = 0 To Len(IDstring) - 3 Step 3  
        ByteStream(i / 3) = CByte(Mid(IDstring, i + 1, 3))  
    Next i  
    vPIDarr = ByteStream  
  
    Set GetObjectFromString = swModExt.GetObjectByPersistReference3((vPIDarr), nRetVal)  
  
    Debug.Assert swPersistReferencedObject_Ok = nRetVal  
    Debug.Assert Not GetObjectFromString Is Nothing
```

End Function

Sub main()

```
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swModExt As SldWorks.ModelDocExtension  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swSelObj As Object  
    Dim swObj As Object  
    Dim vPIDarr As Variant  
    Dim sIDstring As String  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModExt = swModel.Extension
```

```

Set swSelMgr = swModel.SelectionManager
Set swSelObj = swSelMgr.GetSelectedObject6(1, -1): Debug.Assert Not swSelObj Is Nothing
vPIDarr = swModExt.GetPersistReference3(swSelObj): Debug.Assert Not IsEmpty(vPIDarr)
Debug.Print "SelType = " & swSelMgr.GetSelectedObjectType3(1, -1)
If IsEmpty(vPIDarr) Then
    Debug.Print "    ModelDocExtension::GetPersistReference3 broken"
    Exit Sub
Else
    Debug.Print "    ModelDocExtension::GetPersistReference3 exists"
End If
sIDstring = GetStringFromID(swApp, swModel, vPIDarr)
Debug.Print "    Persist Ref = " & sIDstring
Set swObj = GetObjectFromString(swApp, swModel, sIDstring)
If Not Nothing Is swObj Then
    Debug.Assert swSelObj Is swObj
    Debug.Print "    ModelDocExtension::GetObjectByPersistReference3 exists"
Else
    Debug.Print "    ModelDocExtension::GetObjectByPersistReference3 broken"
End If
End Sub

```


Use Persistent Reference Example (VB.NET)

Persistent references are a means of returning to an item between sessions of SOLIDWORKS Design. Persistent references are similar to attributes, but are easier to use. In almost all cases, persistent references can, and should, be used in place of attributes.

This example:

- shows how to use persistent references.
- is primarily diagnostic code that determines whether the currently selected item has a persistent reference from which an object can be retrieved.

```
'-----  
' Preconditions:  
' 1. Open a part, assembly, or drawing.  
' 2. Select an entity.  
'  
' Postconditions:  
' 1. Gets the selected entity type and its PID.  
' 2. Examine the Immediate window.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Function GetStringFromID(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal vPIDarr As  
Object) As String
```

```
        Dim vPID As Object  
        For Each vPID In vPIDarr  
            Debug.Assert(vbByte = VarType(vPID))  
            GetStringFromID = GetStringFromID & Format(vPID, "###000")  
        Next vPID
```

```
    End Function
```

```
    Function GetObjectFromString(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal  
IDstring As String) As Object
```

```
        Dim swModExt As ModelDocExtension  
        Dim ByteStream() As Byte  
        Dim vPIDarr As Object  
        Dim nRetVal As Long  
        Dim i As Long
```

```
        swModExt = swModel.Extension  
        ReDim ByteStream(Len(IDstring) / 3 - 1)
```

```
        For i = 0 To Len(IDstring) - 3 Step 3  
            ByteStream(i / 3) = CByte(Mid(IDstring, i + 1, 3))  
        Next i
```

```
        vPIDarr = ByteStream
```

```
        GetObjectFromString = swModExt.GetObjectByPersistReference3((vPIDarr), nRetVal)
```

```
        Debug.Assert(swPersistReferencedObjectStates_e.swPersistReferencedObject_Ok = nRetVal)  
        Debug.Assert(Not GetObjectFromString Is Nothing)
```

```
    End Function
```

```
    Sub main()
```

```
        Dim swModel As ModelDoc2
```

```

Dim swModExt As ModelDocExtension
Dim swSelMgr As SelectionMgr
Dim swSelObj As Object
Dim swObj As Object
Dim vPIDarr As Object
Dim sIDstring As String

swModel = swApp.ActiveDoc
swModExt = swModel.Extension
swSelMgr = swModel.SelectionManager
swSelObj = swSelMgr.GetSelectedObject6(1, -1) : Debug.Assert(Not swSelObj Is Nothing)

vPIDarr = swModExt.GetPersistReference3(swSelObj) : Debug.Assert(Not IsNothing(vPIDarr))

Debug.Print("SelType = " & swSelMgr.GetSelectedObjectType3(1, -1))

If IsNothing(vPIDarr) Then
    Debug.Print(" ModelDocExtension::GetPersistReference3 broken")
    Exit Sub
Else
    Debug.Print(" ModelDocExtension::GetPersistReference3 exists")
End If

sIDstring = GetStringFromID(swApp, swModel, vPIDarr)

Debug.Print(" Persist Ref = " & sIDstring)

swObj = GetObjectFromString(swApp, swModel, sIDstring)

If Not Nothing Is swObj Then
    Debug.Assert(swSelObj Is swObj)
    Debug.Print(" ModelDocExtension::GetObjectByPersistReference3 exists")
Else
    Debug.Print(" ModelDocExtension::GetObjectByPersistReference3 broken")
End If

End Sub

Public swApp As SldWorks

End Class

```

Use Presentation Transforms to Move Component Example (VBA)

This example shows how to use presentation transforms.

```
'-----  
' Preconditions:  
' 1. Open a fully resolved assembly.  
' 2. Select a component.  
'  
' Postconditions: Moves the component.  
'  
' NOTES:  
' * When the graphics area is redrawn, the display  
'   reverts to how it was at the before running the  
'   macro, that is, the selected component  
'   appears to move back to its original  
'   position. This is by design because  
'   presentation transforms are disabled  
'   at the end of the macro. If presentation transforms  
'   are not disabled, then the component would  
'   remain in its moved position.  
' * When presentation transforms are enabled,  
'   access to most of the menus is blocked.  
'   This is by design because selections cannot  
'   be made because the graphics area is not the  
'   same as the model data.  
'-----
```

Option Explicit

Sub main()

```
    Const MaxSteps As Long = 100  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swAssy As SldWorks.AssemblyDoc  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swComp As SldWorks.Component2  
    Dim vXform As Variant  
    Dim arr(15) As Double  
    Dim swMathUtil As SldWorks.MathUtility  
    Dim swMathXform As SldWorks.MathTransform  
    Dim swModelView as SldWorks.ModelView  
    Dim i As Long  
    Dim rect as Variant  
  
    Set swApp = Application.SldWorks  
    Set swMathUtil = swApp.GetMathUtility()  
    Set swModel = swApp.ActiveDoc  
    Set swAssy = swModel  
    Set swSelMgr = swModel.SelectionManager  
    Set swComp = swSelMgr.GetSelectedObjectsComponent4(1, -1)  
  
    ' Block access to menus  
    swAssy.EnablePresentation = True  
  
    ' Set up unit matrix - no rotation, translation, or scaling  
  
    ' Unit rotation matrix  
    arr(0) = 1#:    arr(1) = 0#:    arr(2) = 0#  
    arr(3) = 0#:    arr(4) = 1#:    arr(5) = 0#  
    arr(6) = 0#:    arr(7) = 0#:    arr(8) = 1#  
  
    ' No translation  
    arr(9) = 0#:    arr(10) = 0#:    arr(11) = 0#  
  
    ' Unit scaling  
    arr(12) = 1#  
  
    ' Not used, so pad with zeros  
    arr(13) = 0#:    arr(14) = 0#:    arr(15) = 0#
```

```

For i = 1 To MaxSteps
    ' Change rotation
    arr(2) = 1.2 * (i / MaxSteps)
    arr(3) = 1.2 * (i / MaxSteps): arr(4) = 0.8 * (i / MaxSteps): arr(5) = 0.9 * (i / MaxSteps)
    arr(6) = 0.8 * (i / MaxSteps)

    ' Change translation
    arr(9) = 0.1 * (i / MaxSteps): arr(10) = 0.2 * (i / MaxSteps)

    ' Should really check that matrix is valid,
    ' especially rotation matrix
    vXform = arr

    Set swMathXform = swMathUtil.CreateTransform((vXform))

    swComp.RemovePresentationTransform
    swComp.PresentationTransform = swMathXform

    ' Redraw so it is shown immediately
    Set swModelView = swModel.ActiveView
    Set rect = Nothing
    swModelView.GraphicsRedraw (rect)
Next i

' Re-enable access to menus
' Comment out to leave component in its moved position
swAssy.EnablePresentation = False

```

End Sub

Use Safe Entity Example (VBA)

This example shows how to use the safe entity functionality.

NOTE: In general, an entity is something that can be selected through the SOLIDWORKS Design user interface. Examples of safe entities include vertices, edges, faces, and features. An entity is useful because you can add an attribute to it to store arbitrary user information. Entities are also useful because they support direct selection through IEntity::Select4. Under normal conditions, an entity is transient and is not valid when the model is rebuilt. Thus, if a rebuild occurs, then the entity must be reacquired. This can be time consuming and prone to errors. Furthermore, it increases the programming complexity. Safe entities are valid across rebuilds of the model. However, this type of entity is not valid across sessions of SOLIDWORKS Design.

```
' -----  
' Preconditions:  
' 1. Open a part, fully resolved assembly, or drawing.  
' 2. Select one of these entities in the graphics area:  
'    * vertex  
'    * edge  
'    * face  
'    * feature  
'  
' Postconditions:  
' 1. Rebuilds the model.  
' 2. Reselects the entity.  
'  
' NOTE: This code does not work for a feature selected  
' in the FeatureManager design tree in a drawing.  
' -----
```

Option Explicit

```
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swSelData As SldWorks.SelectData  
    Dim swEnt As SldWorks.Entity  
    Dim swSafeEnt As SldWorks.Entity  
    Dim bRet As Boolean  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
  
    ' Select safe entity  
    Set swSelMgr = swModel.SelectionManager  
    Set swEnt = swSelMgr.GetSelectedObject6(1, -1)  
    Set swSafeEnt = swEnt.GetSafeEntity  
  
    ' Clear any selections  
    bRet = swModel.ForceRebuild3(False)  
  
    ' Selection still works because entity is safe  
    Set swSelData = swSelMgr.CreateSelectData  
    bRet = swSafeEnt.Select4(True, swSelData)  
  
    Debug.Print "Number of safe entities selected = " & swSelMgr.GetSelectedObjectCount  
  
End Sub
```

Vary Linear Pattern Instances Example (C#)

This example shows how to vary pattern instances.

```
//-----  
// Preconditions:  
// 1. Open public_documents\samples\tutorial\api\box.sldprt.  
// 2. Open the Immediate window.  
  
// Postconditions:  
// 1. Creates a bi-directional linear pattern of a CBORE feature.  
// 2. Specifies to vary pattern instances.  
// 3. Sets the spacing increment of instances in both directions.  
// 4. Modifies the spacing of instance #3.  
// 5. Increments a near countersink diameter for all pattern instances in Direction 1.  
// 6. Modifies the near countersink diameter for instance #3.  
// 7. Inspect the Immediate window and the graphics area.  
//  
// NOTE: Because the part is used elsewhere, do not save changes.  
// -----  
using System;  
using System.Collections.Generic;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
using System.Windows;  
using System.Windows.Forms;  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Diagnostics;  
  
namespace VaryPatternInstances_CSharp  
{  
    public partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            ModelDoc2 swModel;  
            SelectionMgr swSelMgr;  
            Feature feat;  
            LinearPatternFeatureData LPFeatData;  
            bool boolstatus;  
            InstanceToVaryOptions swInstOpts;
```

```

swModel = (ModelDoc2)swApp.ActiveDoc;
swSelMgr = (SelectionMgr)swModel.SelectionManager;

boolstatus = swModel.Extension.SelectByRay(0.0439252690830472, 0.0299751045230323,
0.0126537412306789, -0.400036026779312, -0.515038074910024, -0.758094294050284,
0.000633983676613175, 1, true, 1, 0);

boolstatus = swModel.Extension.SelectByRay(-0.0178361123644777, 0.0298747083165836,
-0.0388627220451099, -0.400036026779312, -0.515038074910024, -0.758094294050284,
0.000633983676613175, 1, true, 2, 0);

boolstatus = swModel.Extension.SelectByID2("CBORE for #6 Binding Head Machine Screw1",
"BODYFEATURE", 0, 0, 0, true, 4, null, 0);
swModel.ActivateSelectedFeature();
swModel.ClearSelection2(true);

boolstatus = swModel.Extension.SelectByID2("CBORE for #6 Binding Head Machine Screw1",
"BODYFEATURE", 0, 0, 0, false, 4, null, 0);

boolstatus = swModel.Extension.SelectByRay(0.0439252690830472, 0.0299751045230323,
0.0126537412306789, -0.400036026779312, -0.515038074910024, -0.758094294050284,
0.000633983676613175, 1, true, 1, 0);

boolstatus = swModel.Extension.SelectByRay(-0.0178361123644777, 0.0298747083165836,
-0.0388627220451099, -0.400036026779312, -0.515038074910024, -0.758094294050284,
0.000633983676613175, 1, true, 2, 0);

Feature swFeat;
FeatureManager swFeatMgr;

swFeatMgr = swModel.FeatureManager;
LinearPatternFeatureData swFeatData;
swFeatData =
(LinearPatternFeatureData)swFeatMgr.CreateDefinition((int)swFeatureNameID_e.swFmLPattern);
swFeatData.D1EndCondition = 0;
swFeatData.D1ReverseDirection = false;
swFeatData.D1Spacing = 0.02;
swFeatData.D1TotalInstances = 2;
swFeatData.D2EndCondition = 0;
swFeatData.D2PatternSeedOnly = false;
swFeatData.D2ReverseDirection = false;
swFeatData.D2Spacing = 0.01;
swFeatData.D2TotalInstances = 2;
swFeatData.GeometryPattern = false;
swFeatData.VarySketch = false;

swFeat = swFeatMgr.CreateFeature(swFeatData);

boolstatus = swModel.Extension.SelectByID2("LPattern1", "BODYFEATURE", 0, 0, 0, false,
0, null, 0);
feat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
LPFeatData = (LinearPatternFeatureData)feat.GetDefinition();

// Turn on Instances To Vary
boolstatus = LPFeatData.AccessSelections(swModel, null);

```

```

LPFeatData.InstancesToVary = true;
Debug.Print("Vary instances? " + LPFeatData.InstancesToVary);

boolstatus = feat.ModifyDefinition(LPFeatData, swModel, null);

// Get and set IInstanceToVaryOptions
boolstatus = LPFeatData.AccessSelections(swModel, null);

swInstOpts = (InstanceToVaryOptions)LPFeatData.GetInstanceToVaryOptions();

// Set D1 and D2 spacing increments of all pattern instances
swInstOpts.D1SpacingIncrement = 0.05;
swInstOpts.D2SpacingIncrement = 0.05;
Debug.Print("D1 spacing increment is " + swInstOpts.D1SpacingIncrement);
Debug.Print("D2 spacing increment is " + swInstOpts.D2SpacingIncrement);

// Get current D1 and D2 modified spacing value for patten instance #3 (2,2)
Debug.Print("Current D1 modified spacing value for instance #3 is " +
swInstOpts.GetD1ModifiedSpacingValue(3));
Debug.Print("Current D2 modified spacing value for instance #3 is " +
swInstOpts.GetD2ModifiedSpacingValue(3));

// Modify the D1 and D2 spacing value for pattern instance #3 (2,2)
boolstatus = swInstOpts.ModifyD1SpacingValue(3,
swInstOpts.GetD1ModifiedSpacingValue(3) + 0.02);
boolstatus = swInstOpts.ModifyD2SpacingValue(3,
swInstOpts.GetD2ModifiedSpacingValue(3) + 0.02);

// Get the new D1 and D2 modified spacing value for pattern instance #3 (2,2)
Debug.Print("New D1 modified spacing value for instance #3 is " +
swInstOpts.GetD1ModifiedSpacingValue(3));
Debug.Print("New D2 modified spacing value for instance #3 is " +
swInstOpts.GetD2ModifiedSpacingValue(3));

// Increment a dimension for all pattern instances in Direction 1
string[] vIncr1Dims = new string[1];
double[] vIncr1DimValues = new double[1];
vIncr1Dims[0] = "Near C'Sink Dia.@Sketch2";
vIncr1DimValues[0] = 0.005;

object incrDims;
object incrDimValues;

swInstOpts.SetD1IncrementDimensions(vIncr1Dims, vIncr1DimValues);
swInstOpts.GetD1IncrementDimensions(out incrDims, out incrDimValues);

// Modify a specific dimension just for pattern instance #3 (2,2)
string[] vDims = new string[1];
double[] vDimValues = new double[1];

```



```

vDims[0] = "Near C'Sink Dia.@Sketch2";
vDimValues[0] = 0.02;
boolstatus = swInstOpts.ModifyDimensions(3, vDims, vDimValues);

swInstOpts.GetModifiedDimensions(3, out incrDims, out incrDimValues);

// Modify the pattern feature with the instance to vary options
LPFeatData.SetInstanceToVaryOptions(swInstOpts);
boolstatus = feat.ModifyDefinition(LPFeatData, swModel, null);
}

// The SldWorks swApp variable is pre-assigned for you.
public SldWorks swApp;

}
}

```

Vary Linear Pattern Instances Example (VB.NET)

This example shows how vary pattern instances.

```
'-----  
'Preconditions:  
' 1. Open public_documents\samples\tutorial\api\box.sldprt.  
' 2. Open the Immediate window.  
  
'Postconditions:  
' 1. Creates a bi-directional linear pattern of a CBORE feature.  
' 2. Specifies to vary pattern instances.  
' 3. Sets the spacing increment of instances in both directions.  
' 4. Modifies the spacing of instance #3.  
' 5. Increments a near countersink diameter for all pattern instances in Direction 1.  
' 6. Modifies the near countersink diameter for instance #3.  
' 7. Inspect the Immediate window and the graphics area.  
  
' NOTE: Because the part is used elsewhere, do not save changes.
```

```
'-----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
  
Partial Class SolidWorksMacro  
    Public Sub main()  
  
        Dim swModel As ModelDoc2  
        Dim swSelMgr As SelectionMgr  
        Dim feat As Feature  
        Dim LPFeatData As LinearPatternFeatureData  
        Dim boolstatus As Boolean  
        Dim swInstOpts As InstanceToVaryOptions  
  
        swModel = swApp.ActiveDoc  
        swSelMgr = swModel.SelectionManager  
  
        boolstatus = swModel.Extension.SelectByRay(0.0439252690830472, 0.0299751045230323,  
0.0126537412306789, -0.400036026779312, -0.515038074910024, -0.758094294050284,  
0.000633983676613175, 1, True, 1, 0)  
  
        boolstatus = swModel.Extension.SelectByRay(-0.0178361123644777, 0.0298747083165836,  
-0.0388627220451099, -0.400036026779312, -0.515038074910024, -0.758094294050284,  
0.000633983676613175, 1, True, 2, 0)  
  
        boolstatus = swModel.Extension.SelectByID2("CBORE for #6 Binding Head Machine Screw1",  
"BODYFEATURE", 0, 0, 0, True, 4, Nothing, 0)
```

```

swModel.ActivateSelectedFeature
swModel.ClearSelection2(True)

boolstatus = swModel.Extension.SelectByID2("CBORE for #6 Binding Head Machine Screw1",
"BODYFEATURE", 0, 0, 0, False, 4, Nothing, 0)

boolstatus = swModel.Extension.SelectByRay(0.0439252690830472, 0.0299751045230323,
0.0126537412306789, -0.400036026779312, -0.515038074910024, -0.758094294050284,
0.000633983676613175, 1, True, 1, 0)

boolstatus = swModel.Extension.SelectByRay(-0.0178361123644777, 0.0298747083165836,
-0.0388627220451099, -0.400036026779312, -0.515038074910024, -0.758094294050284,
0.000633983676613175, 1, True, 2, 0)

Dim swFeat As Object
Dim swFeatMgr As Object

swFeatMgr = swModel.FeatureManager
Dim swFeatData As Object
swFeatData = swFeatMgr.CreateDefinition(swFeatureNameID_e.swFmLPattern)
swFeatData.D1EndCondition = 0
swFeatData.D1ReverseDirection = False
swFeatData.D1Spacing = 0.02
swFeatData.D1TotalInstances = 2
swFeatData.D2EndCondition = 0
swFeatData.D2PatternSeedOnly = False
swFeatData.D2ReverseDirection = False
swFeatData.D2Spacing = 0.01
swFeatData.D2TotalInstances = 2
swFeatData.GeometryPattern = False
swFeatData.VarySketch = False

swFeat = swFeatMgr.CreateFeature(swFeatData)

boolstatus = swModel.Extension.SelectByID2("LPattern1", "BODYFEATURE", 0, 0, 0, False, 0,
Nothing, 0)
feat = swSelMgr.GetSelectedObject6(1, -1)
LPFeatData = feat.GetDefinition

'Turn on Instances To Vary
boolstatus = LPFeatData.AccessSelections(swModel, Nothing)

LPFeatData.InstancesToVary = True
Debug.Print("Vary instances? " & LPFeatData.InstancesToVary)

boolstatus = feat.ModifyDefinition(LPFeatData, swModel, Nothing)

'Get and set IInstanceToVaryOptions
boolstatus = LPFeatData.AccessSelections(swModel, Nothing)

swInstOpts = LPFeatData.GetInstanceToVaryOptions()

```

```

'Set D1 and D2 spacing increments of all pattern instances
swInstOpts.D1SpacingIncrement = 0.05
swInstOpts.D2SpacingIncrement = 0.05
Debug.Print("D1 spacing increment is " & swInstOpts.D1SpacingIncrement)
Debug.Print("D2 spacing increment is " & swInstOpts.D2SpacingIncrement)

'Get current D1 and D2 modified spacing value for patten instance #3 (2,2)
Debug.Print("Current D1 modified spacing value for instance #3 is " &
swInstOpts.GetD1ModifiedSpacingValue(3))
Debug.Print("Current D2 modified spacing value for instance #3 is " &
swInstOpts.GetD2ModifiedSpacingValue(3))

'Modify the D1 and D2 spacing value for pattern instance #3 (2,2)
boolstatus = swInstOpts.ModifyD1SpacingValue(3, swInstOpts.GetD1ModifiedSpacingValue(3) +
0.02)
boolstatus = swInstOpts.ModifyD2SpacingValue(3, swInstOpts.GetD2ModifiedSpacingValue(3) +
0.02)

'Get the new D1 and D2 modified spacing value for pattern instance #3 (2,2)
Debug.Print("New D1 modified spacing value for instance #3 is " &
swInstOpts.GetD1ModifiedSpacingValue(3))
Debug.Print("New D2 modified spacing value for instance #3 is " &
swInstOpts.GetD2ModifiedSpacingValue(3))

'Increment a dimension for all pattern instances in Direction 1
Dim vIncr1Dims(0) As String
Dim vIncr1DimValues(0) As Double
vIncr1Dims(0) = "Near C'Sink Dia.@Sketch2"
vIncr1DimValues(0) = 0.005

swInstOpts.SetD1IncrementDimensions(vIncr1Dims, vIncr1DimValues)
swInstOpts.GetD1IncrementDimensions(vIncr1Dims, vIncr1DimValues)

'Modify a specific dimension just for pattern instance #3 (2,2)
Dim vDims(0) As String
Dim vDimValues(0) As Double
vDims(0) = "Near C'Sink Dia.@Sketch2"
vDimValues(0) = 0.02
boolstatus = swInstOpts.ModifyDimensions(3, vDims, vDimValues)

swInstOpts.GetModifiedDimensions(3, vDims, vDimValues)

'Modify the pattern feature with the instance to vary options
Call LPFeatData.SetInstanceToVaryOptions(swInstOpts)
boolstatus = feat.ModifyDefinition(LPFeatData, swModel, Nothing)

End Sub
''' <summary>

```

```
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks  
End Class
```

Work with Configurations Example (VBA)

This example shows how to work with new and derived configurations.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\2012-sm.sldprt.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates two configurations, Config1 and Config1 Derived.
' 2. Gets whether each of these configurations are derived and the
'    number of children configurations for Config1.
' 3. Gets the number and names of the configurations in the part.
' 4. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim Model As ModelDoc2
Dim ConfigMgr As ConfigurationManager
Dim Cla As Configuration
Dim SelMgr As SelectionMgr
Dim CDerived As Configuration
Dim CParent As Configuration
Dim V As Variant
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks
    Set Model = swApp.ActiveDoc
    Set SelMgr = Model.SelectionManager
    Set ConfigMgr = Model.ConfigurationManager

    ' Create a new configuration named Config1
    ConfigMgr.AddConfiguration2 "Config1", "Config1 comment", "alternateName", 1, "", "no description", True

    ' Create a derived configuration called "Config1 Derived" whose parent configuration is "Config1"
    ConfigMgr.AddConfiguration2 "Config1 Derived", "Config1 Derived Comment", "Alternate Name", 1, "Config1", "no
description", True

    ' Add a rebuild and save mark to all configurations
    bRet = ConfigMgr.AddRebuildSaveMark(2, Nothing)

    ' Display all configuration nodes expanded in the top pane
    ConfigMgr.SetExpanded 0, True

    ' Show Config1 and make it the active configuration
    Model.ShowConfiguration2 ("Config1")

    ' Select the configuration to preview in the bottom pane
    boolstatus = Model.Extension.SelectByID2("Config1", "CONFIGURATIONS", 0, 0, 0, False, 0,
Nothing, 0)
    ConfigMgr.ShowPreview = True

    ConfigMgr.ShowConfigurationDescriptions = True
    ConfigMgr.ShowConfigurationNames = True

    ' Get Config1
    Set Cla = Model.GetActiveConfiguration

    ' Determine if the active configuration is a derived configuration
    Debug.Print Cla.Name & " configuration derived? " & Cla.IsDerived
    Dim VChildren As Variant

    ' Determine the number of children configurations
```

```

Debug.Print "  Number of children configurations: " & Cla.GetChildrenCount

' Get all of the children configurations
VChildren = Cla.GetChildren
Set CDerived = VChildren(0)

' Determine if the active configuration is a derived configuration
Debug.Print CDerived.Name & " configuration derived? " & CDerived.IsDerived

' Get the parent configuration of the derived configuration
Set CParent = CDerived.GetParent

' Determine the number of configurations in this document
Debug.Print "Number of configurations in part: " &
swApp.GetConfigurationCount("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\2012-
sm.sldprt")

' Get the names of these configurations
V = swApp.GetConfigurationNames("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\api\2012-sm.sldprt")

Debug.Print "Names of configurations in part:"
For i = 0 To UBound(V)
    ' Print the names of these configurations
    Debug.Print "  " & V(i)
Next

End Sub

```

Zoom Drawing Sheet to Maximum Size in Window Example (C#)

This example shows how to zoom a drawing sheet to its maximum size within the window.

```
//-----  
// Preconditions: Verify that the drawing to open exists.  
//  
// Postconditions:  
// 1. Opens the drawing.  
// 2. Zooms the drawing sheet to its maximum size within  
//    the window.  
// 3. Examine the graphics area.  
//  
// NOTE: Because the drawing is used elsewhere, do not  
// save changes.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
  
namespace Macro1CSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
  
        public void Main()  
        {  
            ModelDoc2 swModel;  
            ModelDocExtension swModelDocExt;  
            string fileName;  
            int errors = 0;  
            int warnings = 0;  
  
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS  
2018\\samples\\tutorial\\api\\replaceview.slddrw";  
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOption  
s_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);  
            swModelDocExt = (ModelDocExtension)swModel.Extension;  
            swModelDocExt.ViewZoomToSheet();  
  
        }  
  
        /// <summary>  
        /// The SldWorks swApp variable is pre-assigned for you.  
        /// </summary>  
        public SldWorks swApp;  
    }  
}
```


Zoom Drawing Sheet to Maximum Size in Window Example (VBA)

This example shows how to zoom a drawing sheet to its maximum size within the window.

```
'-----  
' Preconditions: Verify that the drawing to open exists.  
'  
' Postconditions:  
' 1. Opens the drawing.  
' 2. Zooms the drawing sheet to its maximum size within  
'    the window.  
' 3. Examine the graphics area.  
'  
' NOTE: Because the drawing is used elsewhere, do not  
' save changes.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim fileName As String  
Dim errors As Long  
Dim warnings As Long  
  
Sub main()  
    Set swApp = Application.SldWorks  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\replaceview.slddrw"  
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING,  
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)  
    Set swModelDocExt = swModel.Extension  
    swModelDocExt.ViewZoomtoSheet  
  
End Sub
```

Zoom_Drawing_Sheet_to_Maximum_Size_in_Window_Example_VBNET (Topic)

SOLIDWORKS API Help

Zoom Drawing Sheet to Maximum Size in Window Example (VB.NET)

This example shows how to zoom a drawing sheet to its maximum size within the window.

```
'-----  
' Preconditions: Verify that the drawing to open exists.  
'-----  
' Postconditions:  
' 1. Opens the drawing.  
' 2. Zooms the drawing sheet to its maximum size within  
'    the window.  
' 3. Examine the graphics area.  
'-----  
' NOTE: Because the drawing is used elsewhere, do not  
' save changes.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System
```

Partial Class SolidWorksMacro

Public Sub main()

```
    Dim swModel As ModelDoc2  
    Dim swModelDocExt As ModelDocExtension  
    Dim fileName As String  
    Dim errors As Integer  
    Dim warnings As Integer
```

```
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\replaceview.slddrw"  
    swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)  
    swModelDocExt = swModel.Extension  
    swModelDocExt.ViewZoomToSheet()
```

End Sub

```
''' <summary>  
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

End Class

Zoom to Region Example (VBA)

This example shows how to zoom to a specific region.

'-----

' Problem:

' This sample code shows how to use:

'

' IModelDoc2::ViewZoomTo2

'

' The input parameters may not be obvious, but
' they are related to the current view orientation.

' The code shows how to transform points
' from model space so that the resulting view is
' zoomed to points selected in model space.

'

' Preconditions:

' (1) Part or assembly is open.

' (2) Two points are picked in the graphics window.

'

' Postconditions:

' Graphics area is zoomed to minimally include
' the two selection points.

'

'-----

Option Explicit

Sub main()

```
Dim swApp          As SldWorks.SldWorks
Dim swModel        As SldWorks.ModelDoc2
Dim swModView      As SldWorks.ModelView
Dim swViewOrient   As SldWorks.MathTransform
Dim swSelMgr       As SldWorks.SelectionMgr
Dim swMathUtil     As SldWorks.MathUtility
Dim vSelPt1        As Variant
Dim vSelPt2        As Variant
Dim swSelPt1       As SldWorks.MathPoint
Dim swSelPt2       As SldWorks.MathPoint
Dim i              As Long
Dim bRet           As Boolean

Set swApp = CreateObject("SldWorks.Application")
Set swMathUtil = swApp.GetMathUtility
Set swModel = swApp.ActiveDoc
Set swSelMgr = swModel.SelectionManager
Set swModView = swModel.ActiveView
Set swViewOrient = swModView.Orientation3
```

```
vSelPt1 = swSelMgr.GetSelectionPoint(1)
```

```
vSelPt2 = swSelMgr.GetSelectionPoint(2)
```

```
Set swSelPt1 = swMathUtil.CreatePoint((vSelPt1))  
Set swSelPt2 = swMathUtil.CreatePoint((vSelPt2))  
Set swSelPt1 = swSelPt1.MultiplyTransform(swViewOrient)  
Set swSelPt2 = swSelPt2.MultiplyTransform(swViewOrient)
```

```
swModel.ViewZoomTo2 _  
    swSelPt1.ArrayData(0), swSelPt1.ArrayData(1), swSelPt1.ArrayData(2), _  
    swSelPt2.ArrayData(0), swSelPt2.ArrayData(1), swSelPt2.ArrayData(2)
```

```
End Sub
```

```
'-----
```